

Aprendizagem por Reforço

Uma Abordagem Centrada em Algoritmos

Versão 1.0

Alexandre Soares

Aprendizagem por Reforço

Uma Abordagem Centrada em Algoritmos

por

Alexandre Soares

Conteúdo

1	Motivação do Trabalho	1
1.1	Objetivo do Estudo	1
2	História do Aprendizado por Reforço	2
3	Introdução ao Aprendizado por Reforço	4
4	Aprendizado por Reforço vs. Outros Paradigmas de Machine Learning	5
4.1	Principais Diferenças	5
4.2	A Evolução Conceitual dos Paradigmas de Aprendizado	6
4.3	Critério de Escolha Entre os Paradigmas de Machine Learning	7
5	Aplicações do Aprendizado por Reforço	8
6	Vantagens e Desvantagens do Aprendizado por Reforço	11
6.1	Vantagens do Aprendizado por Reforço	13
6.2	Desvantagens do Aprendizado por Reforço	14
6.3	Avaliação das Vantagens e Desvantagens	15
7	Interação do Agente com o Ambiente	16
8	Processo de Decisão de Markov	18
8.1	Introdução	18
8.2	Exemplo de um Problema Markoviano	20
8.3	Cadeia de Markov do Problema Proposto	21
8.4	Matriz de Transição do Exemplo Proposto	23
9	Entendendo a Equação de Bellman	25
9.1	Introdução	25
9.2	Ambientes Determinísticos e Estocásticos	26
9.3	Parâmetros que Influenciam a Equação de Bellman	27
9.4	Descrevendo a Equação de Bellman	29
9.5	Equação de Bellman em Ambientes Determinísticos	31
9.6	Restrições e Obstáculos	32
10	Q-Learning sem Exploração do Ambiente	34
10.1	Introdução	34
10.2	Características do Algoritmo Q-Learning sem Exploração	35
10.3	A Equação de Bellman sem Exploração	36
10.4	Funcionamento do Algoritmo Q-Learning sem Exploração	37
10.5	Vantagens do Q-Learning sem Exploração	38
10.6	Limitações do Q-Learning sem Exploração	38
11	Q-Learning com Exploração do Ambiente	39
11.1	Introdução	39
11.2	Funcionamento do Algoritmo Q-Learning	39
11.3	Exploração versus Aproveitamento	41
11.4	Etapas do Algoritmo	43
11.5	Vantagens e Limitações do Q-Learning	44

12 Aplicação Clássica do Aprendizado por Reforço	45
12.1 Descrição Geral do Código	45
12.2 Descrevendo o Ambiente Frozenlake Utilizado	46
12.3 Conhecendo os Valores da Ação Ótima do Ambiente Frozenlake Utilizada	47
12.4 Política Aprendida no Ambiente Frozenlake Utilizada	48
12.5 Episódios com Sucesso e Falha do Ambiente Frozenlake Utilizada	49
12.6 Critérios Usados para a Construção da Política	50
13 Aplicando o Aprendizado por Reforço em Sistemas de Recomendação	51
13.1 Descrição Geral do Código	51
13.2 Distribuição dos Cliques por Segmentos	52
13.3 Comportamento do Click-Through Rate Usado no Treinamento	53
13.4 Decaimento da Exploração ao Longo do Treinamento	54
13.5 Aprendizado da Tabela Q	55
13.6 Resultados Obtidos	56
13.7 Porque Usar um IF/THEN/ELSE não é a Melhor Solução	57
14 Aplicando o Aprendizado por Reforço na Gestão de Energia	58
14.1 Descrição Geral do Código	58
14.2 Aprendizado da Tabela Q	60
14.3 Interpretando os Resultados do Treinamento	62
14.4 Resultado do Treinamento	63
14.5 Conclusão	64
15 Aplicando o Aprendizado por Reforço para Aplicação Financeira	65
15.1 Descrição Geral do Código	65
15.2 Decaimento do Parâmetro de Exploração ϵ	66
15.3 Aprendizado da Tabela Q	67
15.4 Interpretando os Resultados do Treinamento	68
15.5 Evolução do Drawdown ao Longo do Tempo	69
15.6 Desempenho Q-Learning vs Buy & Hold	70
15.7 Conclusão Integrada	70
16 Conclusão do Estudo	71
16.1 Principais Aprendizados	71
Referências	72

Lista de Figuras

2.1	Evolução do Aprendizado por Reforço	2
4.1	Classificação Ampla dos Modelos de Aprendizado	6
4.2	Roteiro de Escolha	7
5.1	Ecosistema de Aplicações do Aprendizado por Reforço	8
6.1	Apresentação do funcionamento do aprendizado por reforço	11
7.1	Interação Agente–Ambiente no Aprendizado por Reforço	16
8.1	Exemplo de uma Cadeia de Markov	21
10.1	Funcionamento Geral do QLearning Sem Exploração	37
11.1	Fluxo de funcionamento do algoritmo Q-Learning.	43
12.1	Mapa do Ambiente Frozenlake Utilizada	46
12.2	Valores Ótimos do Ambiente Frozenlake	47
12.3	Política Aprendida no Ambiente Frozenlake	48
12.4	Episódios de Sucesso e Falha no Ambiente Frozenlake	49
13.1	Distribuição dos Cliques por Segmentos	52
13.2	CTR por Episódio Utilizado no Treinamento	53
13.3	Decaimento do Epsilon ao Longo do Treinamento	54
13.4	Heatmap da Tabela Q Aprendida	55
14.1	Comportamento da Tabela Q Aprendida	60
14.2	Interpretando os Resultados do Treinamento	62
14.3	Resultados do Treinamento	63
15.1	Decaimento do Parâmetro de Exploração	66
15.2	Aprendizado da Tabela Q	67
15.3	Comportamento da Tabela Q Aprendida	68
15.4	Evolução do Drawdown	69

Lista de Tabelas

6.1	Vantagens e Desvantagens do Aprendizado por Reforço	15
11.1	Vantagens e Desvantagens do Ambiente Determinístico	44
13.1	Resultados por Usuário e Segmento	56

1

Motivação do Trabalho

1.1. Objetivo do Estudo

O presente estudo tem como objetivo aplicar e analisar o algoritmo de *Q-Learning* em diferentes contextos, buscando avaliar sua capacidade de aprender políticas ótimas em ambientes markovianos, nos quais o próximo estado depende apenas do estado atual e da ação escolhida.

Pretende-se demonstrar como um agente, por meio da interação contínua com o ambiente, é capaz de maximizar a recompensa acumulada ao longo do tempo. Para isso, o trabalho contempla experimentos em cenários distintos, como ambientes de simulação (*FrozenLake*), gestão de energia e aplicações financeiras.

O trabalho também tem a intenção de evidenciar a flexibilidade do algoritmo de *Q-Learning* em lidar com diferentes tipos de ambientes. Isso significa mostrar sua aplicabilidade tanto em cenários simples, como jogos ou ambientes de simulação, quanto em contextos mais complexos, como gestão de energia ou sistemas financeiros. Essa característica reforça a robustez do método e sua capacidade de adaptação, permitindo que seja explorado em diferentes domínios de problemas de tomada de decisão sequencial.

Outro aspecto relevante é a análise dos benefícios e limitações do *Q-Learning*, especialmente no que diz respeito à convergência, à escolha dos hiperparâmetros e à escalabilidade. Além de destacar esses pontos, o estudo também busca mostrar como a técnica pode servir de base para cenários ainda mais sofisticados, principalmente quando combinada com métodos avançados como o *Deep Q-Learning*. Essa integração amplia o alcance do algoritmo, tornando-o aplicável a problemas de maior dimensão e complexidade, ao mesmo tempo em que expande seu potencial para ambientes reais e dinâmicos.

Assim, o estudo tem como meta validar e analisar a eficácia do *Q-Learning* não apenas em termos de retorno, mas também considerando a resiliência da estratégia diante de riscos e incertezas presentes no ambiente.

2

História do Aprendizado por Reforço

Os princípios do aprendizado por reforço estão profundamente conectados à evolução da inteligência artificial, à neurociência e à teoria do controle. É considerado um dos paradigmas mais antigos de aprendizagem; contudo, também se destaca como um dos mais promissores e dinâmicos na atualidade, especialmente após a introdução das redes neurais profundas. Essa evolução é apresentada na Figura-2.1.

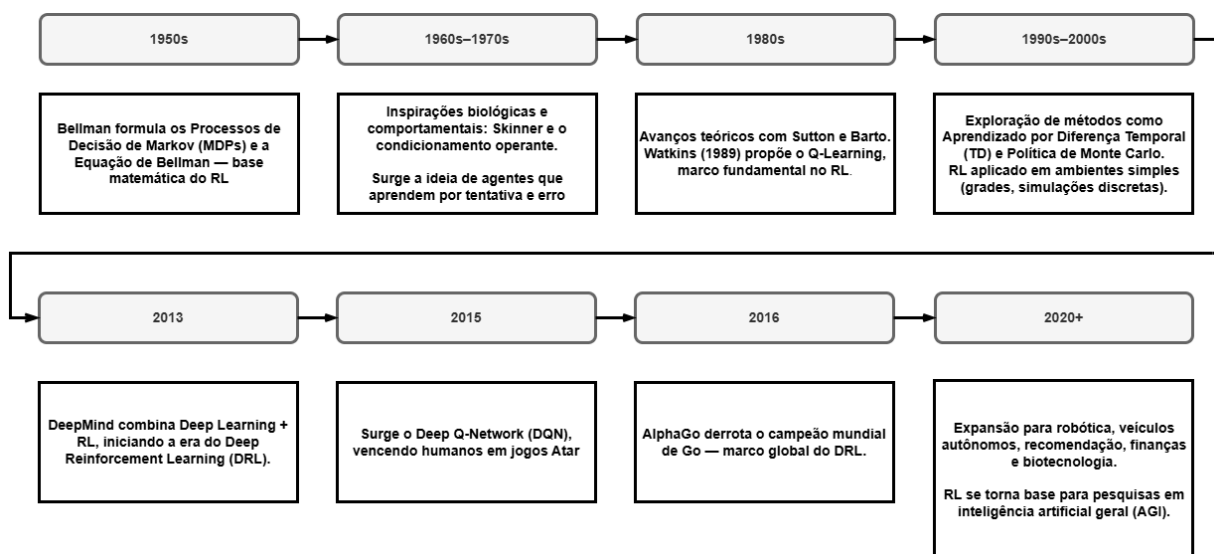


Figura 2.1: Evolução do Aprendizado por Reforço

O princípio básico do aprendizado por reforço surgiu nas décadas de 1950 e 1960, influenciado por conceitos comportamentais da psicologia e da biologia. A concepção de um agente que adquire conhecimento por meio de tentativa e erro origina-se de experimentos com animais, tais como os renomados testes de condicionamento realizados por Skinner [1]. Esse caminho motivou os especialistas em ciência da computação a criar programas de computador que conseguem aprender ao interagir com diferentes ambientes, dispensando qualquer supervisão direta.

Sob uma perspectiva técnica, os modelos formais iniciais de aprendizado por reforço surgiram com a introdução dos processos de decisão de Markov, apresentados por Richard Bellman na década de 1950 [2]. Bellman elaborou a equação que tem o seu nome, a qual se firmou como base para os algoritmos modernos de aprendizado por reforço. Nas décadas seguintes, algoritmos como programação dinâmica, aprendizado por diferença temporal e Q-learning foram propostos e analisados de forma acadêmica.

Investigadores como Andrew Barto, Richard Sutton e Chris Watkins, na década de 1980, possibilitaram avanços significativos tanto teóricos quanto práticos neste campo [7]. O Q-learning, um algoritmo criado por Watkins em 1989, é um progresso significativo ao possibilitar que os agentes obtenham conhecimento sem a necessidade de um modelo explícito do ambiente. Sutton, um dos coautores da obra seminal "Reinforcement Learning: An Introduction", contribuiu significativamente para a organização e divulgação desse domínio perante a comunidade científica.

Todavia, ao longo de muitos anos, o aprendizado por reforço enfrentou um desafio substancial: a alta dimensionalidade dos espaços de estados e ações. Os métodos antigos funcionavam bem apenas em situações específicas e em quantidades pequenas. A superação dessa dificuldade começou nos anos 2010. Foi quando as melhorias em aprendizado profundo se misturaram ao aprendizado por reforço. Essa combinação deu origem à Aprendizagem por Reforço Profundo.

A revolução do Aprendizado por Reforço Profundo (DRL) teve seu início em 2013, quando a startup britânica DeepMind apresentou um agente capaz de aprender a jogar jogos do Atari diretamente a partir dos pixels exibidos na tela, empregando apenas recompensas como indicativo de aprendizado [6].

Em 2015, este grupo lançou o Deep Q-Network (DQN), um modelo capaz de superar jogadores humanos em diversos jogos. No ano de 2016, o mundo presenciou o reconhecimento do agente AlphaGo, criado pela DeepMind, sobre o campeão mundial de Go, um jogo famoso pela sua complexidade e amplo campo de busca.

Desde então, o Aprendizado por Reforço se firmou como uma das áreas mais ágeis da inteligência artificial, demonstrando aplicações que vão além dos jogos, abrangendo robótica, sistemas de recomendação, veículos autônomos, administração de tráfego, setores financeiros e biotecnologia. Ele representa o paradigma mais compatível com a ideia de inteligência geral artificial, pois abrange a interação contínua com o ambiente, a realização de decisões sequenciais e o aprendizado a longo prazo.

Atualmente, o aprendizado por reforço é uma área de investigação em instituições de educação superior, centros de pesquisa e organizações ao redor do mundo. Apesar dos desafios que ainda estão presentes nos dias atuais, essa área continua a ser uma das mais fascinantes e promissoras no domínio da inteligência artificial.

Introdução ao Aprendizado por Reforço

A aprendizagem por reforço representa uma metodologia que não faz uso de dados pré-estabelecidos. A máquina captura dados do ambiente e, por meio de recompensas (feedbacks), adquire a habilidade de realizar ações que geram os resultados mais favoráveis. Esse processo de aprendizado é alcançado através do retorno proporcionado pelo ambiente e pela maximização dessas recompensas, ajustando-se progressivamente para otimizar os ganhos.

Esta metodologia se fundamenta na experimentação ativa e no retorno positivo. As medidas adotadas podem produzir efeitos duradouros, afetando não somente o cenário atual, mas também as escolhas futuras. A complexidade está em balancear a busca por novas oportunidades com a utilização estratégica do conhecimento já estabelecido.

A questão é estruturada como um processo de decisão Markoviano [3], no qual o agente analisa estados, seleciona ações e obtém recompensas. O propósito consiste em adquirir uma função que determina a ação a ser executada em cada condição, a fim de otimizar a recompensa total esperada ao longo do tempo. Essa metodologia é eficaz por analisar decisões sequenciais em cenários incertos de forma abrangente, diferentemente de abordagens que se concentram em partes isoladas.

Esta metodologia conecta diversos campos do conhecimento humano, como matemática, engenharia, ciências cognitivas, se assemelhando aos processos da psicologia e neurociência, como os mecanismos de recompensa mediados pela dopamina no cérebro. Assim, o aprendizado por reforço surge tanto como uma ferramenta computacional eficaz quanto como um modelo teórico promissor para entender comportamentos inteligentes em humanos e máquinas.

4

Aprendizado por Reforço vs. Outros Paradigmas de Machine Learning

4.1. Principais Diferenças

O Aprendizado por Reforço se distingue de outros modelos de aprendizado de máquina, tanto pela forma como obtém conhecimento quanto pela natureza dos problemas que trata. Ao contrário dos métodos supervisionados, que se baseiam em exemplos categorizados, e dos métodos não supervisionados, que exploram estruturas ou padrões em dados sem rótulos, o aprendizado por reforço baseia-se na interação sequencial entre um agente e seu ambiente, onde o agente faz escolhas e adquire conhecimento com base nas repercussões de suas ações.

O objetivo principal do Aprendizado por Reforço é otimizar uma função de recompensa acumulada ao longo do tempo, isto é realizado por meio do aprendizado que seleciona as ações que orientam a situações mais vantajosas. Esta abordagem é baseada nas dinâmicas de aprendizagem tanto de animais quanto de humanos, sendo formalmente estabelecida através dos Processos de Decisão de Markov.

Uma característica fundamental do Aprendizado por Reforço é o dilema entre exploração e aproveitamento: o agente deve investigar ações não familiarizadas para entender seus efeitos, enquanto, ao mesmo tempo, utiliza o conhecimento previamente adquirido para otimizar o retorno. Essa dinâmica torna o aprendizado por reforço adequado para questões em que o retorno não ocorre de forma imediata, mas sim de maneira vagarosa, e as escolhas realizadas impactam diretamente os estados futuros, como em jogos, controle de robôs, sistemas de recomendação e navegação autônoma.

4.2. A Evolução Conceitual dos Paradigmas de Aprendizado

Com os estudos de Yuxi Li [4], podemos ampliar a organização anterior, como apresentado na Figura 4.1 e assim estabelecer uma conexão entre aprendizado por reforço profundo, aprendizado profundo, aprendizado por reforço, aprendizado supervisionado, aprendizado não supervisionado, aprendizado de máquina e inteligência artificial, refletindo uma hierarquia conceitual e tecnológica em constante evolução.

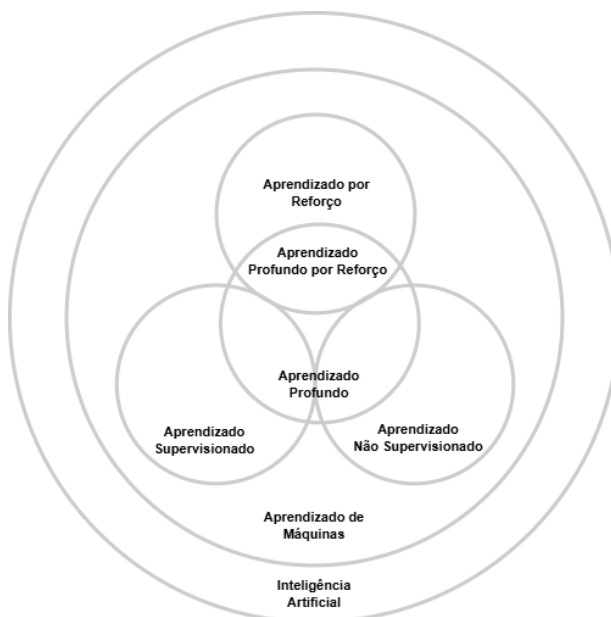


Figura 4.1: Classificação Ampla dos Modelos de Aprendizado

A inteligência artificial é um vasto campo que engloba uma variedade de métodos voltados para a imitação da inteligência humana. O aprendizado de máquina é um subcampo voltado para a criação de algoritmos que obtêm conhecimento através da análise de dados.

O aprendizado de máquina é constituído por três paradigmas fundamentais: o aprendizado supervisionado, onde o modelo adquire o conhecimento utilizando exemplos rotulados; o aprendizado não supervisionado, que busca identificar padrões ocultos sem a utilização de rótulos; e o aprendizado por reforço, onde um agente desenvolve sua capacidade por meio da interação com o ambiente e da obtenção de recompensas.

O aprendizado por reforço profundo integra os princípios do aprendizado profundo ao aprendizado por reforço, permitindo que os agentes determinem a ação mais adequada em ambientes complexos e de alta dimensionalidade. Tanto o aprendizado profundo quanto o aprendizado por reforço profundo situam-se na vanguarda da inteligência artificial, uma vez que tratam de diversos problemas tradicionais dessa área, tais como o reconhecimento de padrões, a tomada de decisões em tempo real e o controle em ambientes dinâmicos.

4.3. Critério de Escolha Entre os Paradigmas de Machine Learning

Esse critério é apresentado na Figura-4.2 e serve como uma orientação inicial para definir o paradigma mais adequado de aprendizado de máquina, facilitando a escolha entre técnicas supervisionadas, não supervisionadas, por reforço ou alternativas.

Se o problema possuir dados históricos com rótulos claros que permitem realizar previsões ou classificações, o caminho ideal é o Aprendizado Supervisionado. Essa abordagem é apropriada quando os exemplos de entrada estão associados a saídas esperadas, como identificar se um e-mail é spam ou não, ou prever o valor de uma casa com base em suas características.

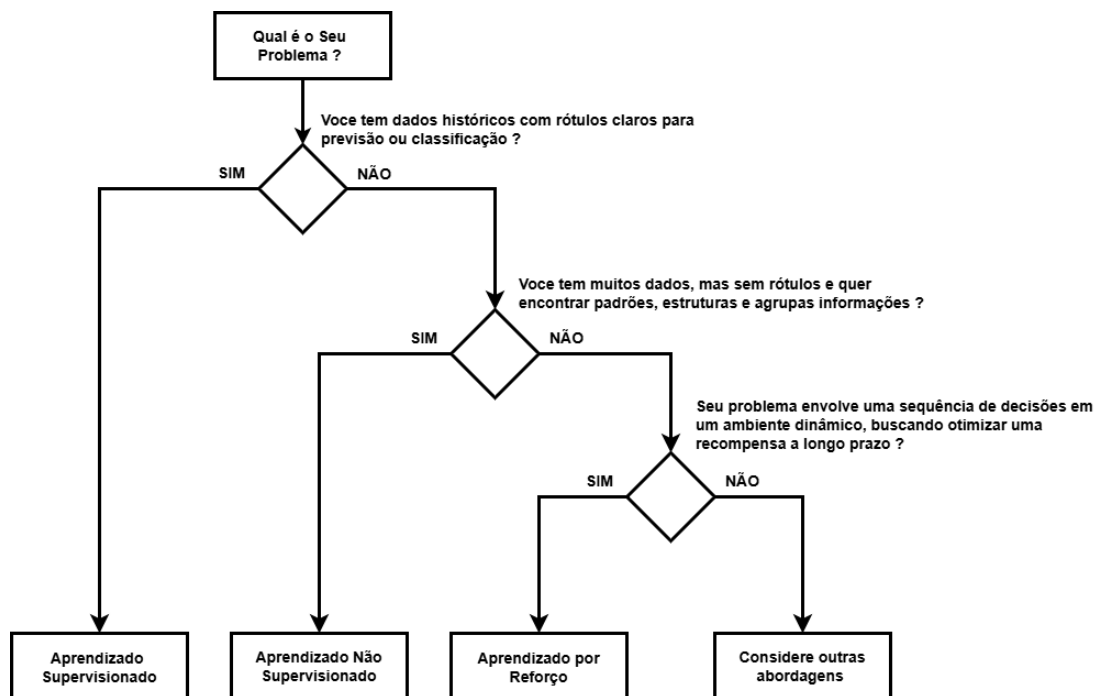


Figura 4.2: Roteiro de Escolha

Se não há rótulos disponíveis, a próxima consideração é se você possui muitos dados e deseja encontrar padrões, estruturas ou agrupamentos de forma autônoma. Nesse caso, a recomendação é o Aprendizado Não Supervisionado, útil para tarefas como agrupamento de clientes ou redução de dimensionalidade.

Por outro lado, se não há rótulos e o objetivo é tomar decisões sequenciais em um ambiente dinâmico visando maximizar uma recompensa a longo prazo, a melhor escolha é o Aprendizado por Reforço. Essa abordagem é aplicada em contextos como jogos, controle de robôs ou sistemas de recomendação com múltiplas etapas.

Caso nenhuma dessas condições seja satisfeita, recomenda-se considerar outras abordagens, que podem envolver heurísticas, algoritmos evolutivos, lógica simbólica ou sistemas especialistas, a depender da natureza e das restrições do problema.

5

Aplicações do Aprendizado por Reforço

O Aprendizado por Reforço tem demonstrado aplicações inovadoras em diversos setores da economia e da tecnologia, conforme ilustrado na Figura-5.1 [5], evidenciando sua versatilidade e eficácia na solução de problemas complexos associados à tomada de decisão sequencial.

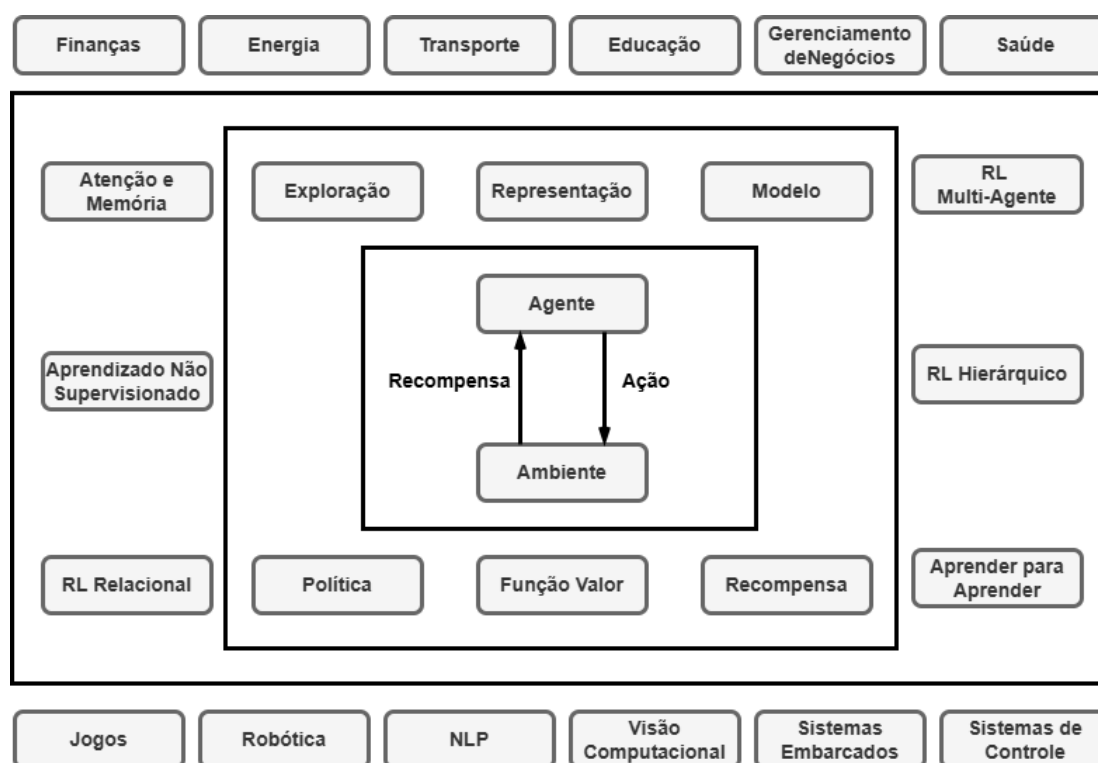


Figura 5.1: Ecosistema de Aplicações do Aprendizado por Reforço

Na indústria financeira, o aprendizado por reforço transforma a maneira como são feitas as negociações, isto porque atualmente estas operações são realizadas por meio de negociações algorítmicas, permitindo assim que os agentes melhorem os portfólios de investimentos, pois respondem de forma dinâmica às condições do mercado.

A detecção de fraudes baseadas em aprendizagem por reforço tornou mais eficiente a identificação de uma fraude, uma vez que os algoritmos se adaptam às novas estratégias dos fraudadores, permitindo assim às instituições uma reação mais rápida e, assim, reduzir suas perdas.

Atualmente, os algoritmos administram os riscos e recursos financeiros por meio de dados históricos e, assim, são capazes de aprender a partir de retroalimentações históricas e modificar suas estratégias em tempo real.

No campo da energia, ajuda a aperfeiçoar as redes elétricas inteligentes, harmonizando oferta e demanda de maneira autônoma. Estas contribuições também são observadas na gestão energética de condomínios, seja de casas ou edifícios, onde esses modelos aprendem os padrões de consumo e regulam de maneira automática a climatização e a iluminação. A orientação e a operação de turbinas eólicas e painéis solares podem ser aprimoradas por meio de agentes que aprendem com as variações das condições climáticas.

O segmento de transporte observa aplicações significativas em automóveis autônomos, nos quais o aprendizado por reforço possibilita que os veículos adquiram habilidades para se deslocar em cenários complexos e imprevisíveis. As contribuições destes algoritmos também estão presentes nos sistemas de gerenciamento de tráfego urbano que buscam otimizar semáforos e o fluxo de veículos, o que contribui para a diminuição de congestionamentos. Veículos aéreos não tripulados de entrega e robôs logísticos também utilizam aprendizado por reforço para o planejamento de rotas eficazes.

Na educação, observamos grandes contribuições dos sistemas adaptativos que buscam personalizar a experiência da aprendizagem, uma vez que modulam a dificuldade bem como o conteúdo conforme o progresso de cada estudante. Professores virtuais inteligentes empregam o aprendizado por reforço para identificar os momentos e as maneiras de intervir no processo educativo, otimizando a retenção do conhecimento.

Na gestão das empresas, os ganhos de escalabilidade operacional são notáveis a partir do uso de sistemas de recomendação, os quais aprendem as preferências dos clientes à medida que os modelos interagem com eles. Estes recursos são rotineiros nos dias atuais e estão disponíveis nos Chatbots e assistentes virtuais que melhoram constantemente suas respostas em função do retorno recebido dos usuários.

A gestão dos inventários de produtos também é impactada com estes benefícios do aprendizado por reforço, isto porque estes modelos permitem a otimização das cadeias de suprimento equilibrando a demanda com a renovação dos estoques e assim otimizando o capital investido.

Na área da saúde, o aprendizado por reforço permite que os tratamentos médicos sejam alterados quando observada a melhora dos pacientes, ou seja, a dose de medicamentos pode ser alterada de modo automático, preservando a saúde e os custos médicos. Sistemas de diagnóstico adquirem conhecimento a partir de cada ocorrência, aprimorando sua exatidão, tornando-se assim um importante instrumento de cooperação com os profissionais da saúde.

A indústria de jogos é o campo mais evidente das contribuições do aprendizado por reforço, onde sistemas como AlphaGo e OpenAI Five demonstram habilidades superiores às humanas. Esses êxitos não se limitam a melhorar a experiência de entretenimento, mas funcionam como laboratórios para a criação de novas técnicas que podem ser aplicadas a problemas do mundo real.

A robótica está em pleno crescimento, muito se deve ao aprendizado por reforço, isto porque as competências complexas requeridas para estes equipamentos, tais como: manipulação de objetos, navegação em ambientes desconhecidos e controle de movimentos visando eficiência e exatidão só poderiam ter sido alcançadas por meio da interação com o ambiente.

O processamento de linguagem natural utiliza o aprendizado por reforço para melhorar os sistemas de tradução e a produção de textos, tornando mais fluidos os textos produzidos. Este sucesso é baseado no retorno dos usuários, o que melhora os sistemas de diálogo.

Na visão computacional, o uso de câmeras inteligentes permitiu o desenvolvimento da capacidade de foco automático, uma vez que os ajustes nas configurações suportados por sistemas baseados em reforço foram utilizados para aprender e, posteriormente, eliminar os diversos ajustes anteriormente manuais, e criando assim um novo mercado.

Os sistemas embarcados e de controle incorporam o aprendizado por reforço com o objetivo de melhorar a otimização do desempenho em tempo real, ajustando-se a alterações nas condições operacionais sem requerer reprogramação manual.

As principais aplicações do aprendizado por reforço englobam elementos tais como a função de valor, responsável por avaliar o retorno futuro de ações, as políticas, que estabelecem diretrizes para a tomada de decisões, e os sistemas de recompensa, que dirigem o processo de aprendizagem.

Um elemento comum em todas as aplicações anteriormente apresentadas está no requerimento de aprender e se adaptar por meio da experiência, realizando decisões otimizadas em contextos dinâmicos e incertos, nos quais soluções pré-programadas se revelariam inadequadas ou inviáveis de serem implementadas.

6

Vantagens e Desvantagens do Aprendizado por Reforço

A Figura-6.1 ilustra de forma didática o funcionamento geral do aprendizado por reforço como um processo de transformação de dados brutos em decisões organizadas e otimizadas, por meio da interação contínua entre agente e ambiente.

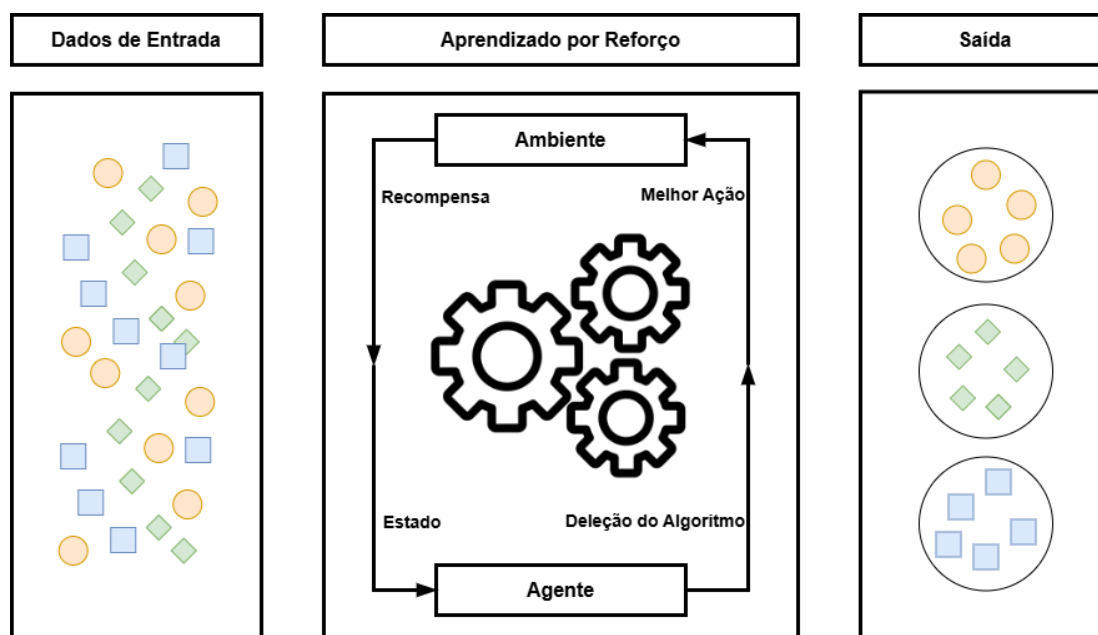


Figura 6.1: Apresentação do funcionamento do aprendizado por reforço

No lado esquerdo, encontram-se os dados de entrada, ilustrados por formas geométricas variadas que representam distintas tipologias ou categorias de informações não estruturadas e heterogêneas. Essas informações são inseridas no sistema desprovidas de qualquer organização anterior, demandando que o algoritmo elabore estratégias para interpretá-las e processá-las de forma eficaz.

O núcleo do aprendizado por reforço é simbolizado pelo centro da imagem, evidenciando a interação entre dois componentes essenciais: o agente e o ambiente. O agente tem a incumbência de realizar escolhas fundamentadas nas condições do ambiente, efetuando ações que são julgadas de acordo com a recompensa obtida. A recompensa ou feedback atua como um indicativo de retorno, direcionando o agente acerca da eficácia de suas escolhas. Com base nesse ciclo de observação, ação e recompensa, o agente ajusta sua política de decisão, visando determinar a ação mais eficaz para otimizar os resultados ao longo do tempo.

O processo iterativo é representado pelas engrenagens centrais, que é um sistema de aprendizado contínuo, no qual o conhecimento é adquirido, adaptado e aprimorado a cada nova interação. A designação "Deleção do Algoritmo" diz respeito ao ciclo de atualização interna das estratégias do agente, ou seja, ao procedimento de aprimoramento gradual fundamentado nas vivências anteriores.

À direita, observa-se a saída. As informações que anteriormente estavam desorganizadas estão agora classificadas corretamente em três grupos distintos, onde cada grupo contém unicamente um tipo de figura geométrica. Este resultado demonstra a habilidade do aprendizado por reforço em adquirir o conhecimento ou a capacidade de realizar decisões eficazes em contextos não familiares.

Assim, a ilustração representa o fluxo integral do aprendizado por reforço: desde a coleta de dados primários, através da interação cíclica entre o agente e o ambiente para o aprendizado e a otimização de estratégias, até a geração de resultados estruturados que demonstram o comportamento adquirido. Refere-se a uma apresentação eficiente da lógica subjacente a algoritmos que são capazes de aprender a atuar, ajustando-se continuamente com base na experiência adquirida.

6.1. Vantagens do Aprendizado por Reforço

Uma das maiores vantagens do aprendizado por reforço é a sua capacidade de adaptação em ambientes com elevada incerteza. Os agentes são projetados para aprender de maneira contínua, e durante o processo de descoberta ajustam suas estratégias com base nas recompensas ou penalidades que recebem do ambiente. Assim, sistemas baseados em regras fixas tornam-se ultrapassados por essa abordagem que é mais flexível e permite generalizar o conhecimento obtido para ser usado em novas situações.

Este aprendizado é particularmente bem-sucedido na tomada de decisões sequenciais. Em muitos problemas do mundo real, uma única decisão afeta as decisões futuras, e o objetivo é otimizar uma sequência de ações ao longo do tempo. Pense em um jogo de xadrez: cada movimento influencia os movimentos subsequentes, e a vitória depende de uma série de decisões bem pensadas. O aprendizado por reforço se destaca aqui porque seu objetivo é maximizar a recompensa cumulativa a longo prazo, em vez de apenas a recompensa imediata.

No processo de descoberta, o agente produz seus próprios dados quando interage com o ambiente por meio de recompensas e penalizações, tornando-se uma opção quando a rotulação de dados é complexa ou inviável, por exemplo, em automação de sistemas industriais complexos.

6.2. Desvantagens do Aprendizado por Reforço

Problemas considerados simples não são indicados o uso do aprendizado por reforço, uma vez que o método foi concebido para operar em cenários onde são requeridas decisões sequenciais e adaptação dinâmica. Assim, a utilização em atividades triviais pode resultar em um desperdício de recursos computacionais.

Assim, um problema simples onde o método não é indicado seria aquele onde a complexidade do método não traz benefícios se consideradas as abordagens mais simples e eficientes. Algumas considerações devem ser feitas antes de selecionar o aprendizado por reforço. Existe uma regra clara e direta para chegar à resposta correta, por exemplo, realizar uma classificação de imagens ou reconhecer padrões básicos. O resultado de cada ação é imediato, não dependendo das decisões anteriores, ou seja, não existe uma cadeia de consequências a ser considerada. O espaço de todos os estados e ações é pequeno e completamente conhecido, e, como consequência, todas as possibilidades são facilmente mapeadas.

Os modelos de aprendizado por reforço exigem extensos volumes de dados para a fase de treinamento, o que resulta em elevados custos computacionais e em um prolongado tempo de processamento. Esta demanda se deve a exigência de investigar e adquirir conhecimentos a partir do ambiente, isto implica que o agente realize múltiplas iterações simulando milhões de interações para obter o comportamento ideal.

Em contextos como automóveis autônomos ou sistemas robóticos, surgem desafios suplementares concernentes à preservação dos resultados alcançados, tanto no que se refere ao hardware quanto ao software. Os gastos financeiros direcionados à manutenção e atualização podem ser consideráveis; em virtude dessa característica, sua utilização em larga escala é obstaculizada.

Outro aspecto fundamental é que a prática excessiva pode resultar na sobrecarga dos estados do modelo, comprometendo sua habilidade de generalização e inferência. O problema é piorado por um consumo excessivo de memória, prejudicando o desempenho do modelo e podendo inviabilizar sua operação em ambientes com recursos limitados.

Assim, apesar de ser extremamente eficaz em aplicações complexas, a utilização do aprendizado por reforço demanda uma análise minuciosa da viabilidade técnica e econômica, a fim de assegurar que sua aplicação seja justificável em relação às exigências do problema em foco.

6.3. Avaliação das Vantagens e Desvantagens

As seções 6.1 e 6.2 forneceram uma análise detalhada e aprofundada sobre os diversos aspectos positivos e negativos relacionados ao aprendizado por reforço. Dado que esse assunto é bastante complexo, é essencial consolidar as principais conclusões obtidas.

Nesse sentido, a Tabela-6.1 foi elaborada com o intuito de apresentar os pontos mais relevantes de forma clara e objetiva, facilitando, assim, a comparação entre as diferentes características abordadas, e permitindo uma conclusão mais sólida sobre o tema em questão.

Tabela 6.1: Vantagens e Desvantagens do Aprendizado por Reforço

Vantagens	Desvantagens
É capaz de resolver problemas de alta complexidade, oferecendo soluções com elevado grau de precisão	Não é adequado para problemas simples, pois foi concebido para lidar com situações mais complexas
Sua precisão advém da semelhança com o processo de aprendizado humano.	Seu uso em tarefas simples resulta em desperdício de recursos computacionais.
O modelo passa por um treinamento rigoroso, que ajuda a corrigir erros ao longo do tempo.	Requer grandes volumes de dados para treinamento, o que implica em alto custo computacional e consumo de tempo.
Pode ser integrado a redes neurais, originando o deep reinforcement learning.	A aplicação em casos do mundo real possui alto custo de manutenção (hardware e software), como em veículos autônomos e robôs.
Por aprender continuamente, tende a evitar a repetição de erros anteriores.	O treinamento excessivo pode sobrecarregar os estados do modelo, prejudicando sua capacidade de inferência.
Permite a construção de diversos modelos voltados à resolução de problemas específicos.	O consumo exagerado de memória durante o processamento de dados pode comprometer a performance do modelo.
É amplamente aplicado na criação de simuladores, detecção de objetos em veículos autônomos, robótica, entre outros.	—
Mesmo na ausência de dados de treinamento explícitos, o modelo aprende a partir da experiência acumulada.	—
Oferece modelos eficazes para problemas complexos que, para humanos, seriam desafiadores.	—

Interação do Agente com o Ambiente

O aprendizado por reforço baseia-se em um conjunto de elementos que estabelecem a interação entre um agente e seu ambiente ao longo do tempo [7]. Todos esses elementos exercem uma função fundamental no processo de tomada de decisão sequencial; este ambiente é mostrado na Figura-7.1.

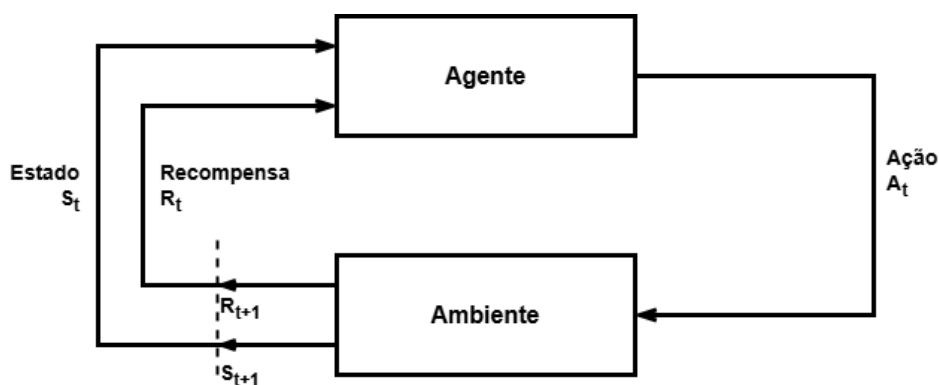


Figura 7.1: Interação Agente–Ambiente no Aprendizado por Reforço

O agente representa a entidade encarregada de realizar decisões e adquirir conhecimentos a partir da experiência. O agente estabelece uma interação com um ambiente, que é compreendido como tudo aquilo que está além do controle direto do agente. A cada momento, o agente analisa uma condição do ambiente, seleciona uma ação e, como resultado, o ambiente muda para um novo estado, além de oferecer uma recompensa, caracterizando assim um indicativo numérico que revela a qualidade da ação executada.

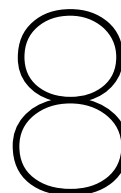
A recompensa ou feedback se estabelece como um critério primordial de avaliação utilizado pelo agente para adquirir conhecimento. A finalidade da aprendizagem por reforço busca maximizar a recompensa total obtida ao longo do tempo, o que requer a consideração não apenas de benefícios imediatos, mas também das consequências a longo prazo de cada escolha.

Para decidir qual ação adotar, o agente utiliza uma política (π), que basicamente se configura como uma função destinada a associar a cada estado uma distribuição de probabilidades correspondente às ações disponíveis. A política pode ser caracterizada como determinística quando, visando a um estado específico, sempre opta pela mesma ação, ou como estocástica, quando confere probabilidades a diferentes ações.

Um aspecto fundamental da aprendizagem consiste na aplicação das funções de valor, as quais avaliam o retorno esperado. A função de valor de estado ($v_\pi(s)$) avalia a recompensa futura esperada ao iniciar no estado s e adotar a política π ; por sua vez, a função de valor de ação ($q_\pi(s, a)$) calcula a recompensa esperada ao realizar a ação a no estado s e prosseguir seguindo a política π a partir desse momento.

Um dos principais obstáculos enfrentados reside na dicotomia entre Exploração e Aproveitamento: o agente deve realizar investigações sobre novas ações para identificar aquelas que proporcionam recompensas satisfatórias, ao mesmo tempo em que precisa empregar o conhecimento adquirido para garantir recompensas consistentes. A conciliação entre essas duas posturas é essencial para um aprendizado eficaz.

Esses elementos interagem de forma cíclica, ou seja, o agente analisa o ambiente, seleciona ações de acordo com sua política, revisa suas estimativas de valor fundamentadas na recompensa e nos estados analisados, e modifica sua política visando aprimorar seu desempenho ao longo do tempo.



Processo de Decisão de Markov

8.1. Introdução

O Processo de Decisão de Markov, conhecido pela sigla MDP, representa uma ferramenta matemática fundamental para modelar situações onde precisamos tomar decisões sequenciais em ambientes incertos; este processo nos ajuda a encontrar as melhores estratégias de ação ao longo do tempo.

Imagine um robô navegando por um labirinto, um investidor decidindo quando comprar e vender ações, ou um médico escolhendo tratamentos para um paciente. Em todos esses cenários, é compartilhada uma característica comum: existe uma sequência de decisões onde cada escolha afeta as opções e resultados futuros, e é exatamente esse tipo de problema que os MDPs foram desenvolvidos para resolver de forma sistemática e otimizada.

A essência de um MDP reside na ideia de que, a qualquer momento, nosso sistema se encontra em um determinado estado, e a partir desse estado podemos escolher entre diferentes ações possíveis. Cada ação escolhida nos leva a um novo estado de acordo com certas probabilidades, e ao fazer essa transição, recebemos uma recompensa que pode ser positiva ou negativa. O objetivo fundamental é aprender ou descobrir uma política - essencialmente um plano de ação - que nos diga qual ação tomar em cada estado possível, de forma a maximizar o total de recompensas que receberemos ao longo do tempo.

O que torna os MDPs especialmente poderosos e matematicamente tratáveis é uma propriedade chamada propriedade de Markov, que estabelece que tudo o que precisamos saber para tomar a melhor decisão no momento presente está contido no estado atual.

Em outras palavras, não precisamos nos preocupar com toda a história de como chegamos ao estado atual - apenas conhecer onde estamos agora é suficiente para tomar a decisão ótima. Esta propriedade pode parecer uma simplificação, mas, na verdade, é surpreendentemente realista para muitos problemas práticos e permite que desenvolvamos algoritmos eficientes para encontrar soluções ótimas.

A resolução de um MDP envolve encontrar valores para cada estado que representem o quão desejável é estar naquele estado, considerando todas as recompensas futuras que podemos esperar obter a partir dali.

Estes valores são calculados levando em conta não apenas as recompensas imediatas que podemos receber, mas também as recompensas futuras, geralmente aplicando um fator de desconto que faz com que recompensas mais distantes no tempo tenham menor peso na decisão atual. Uma vez que temos esses valores calculados, determinar a política ótima torna-se relativamente direta: em cada estado, simplesmente escolhemos a ação que nos leva ao estado com maior valor esperado.

Os MDPs encontram aplicações práticas em uma ampla variedade de domínios, desde jogos e entretenimento até problemas críticos de engenharia e negócios. Na área de jogos, são utilizados para criar inteligência artificial que pode competir em jogos de estratégia como xadrez ou Go.

Em robótica, ajudam robôs a navegar em ambientes complexos e realizar tarefas de manipulação. Na área financeira, são aplicados em algoritmos de trading automático e gestão de carteiras de investimento. Na medicina, auxiliam no desenvolvimento de protocolos de tratamento que se adaptam à resposta individual de cada paciente.

Uma das grandes vantagens dos MDPs é que eles fornecem tanto uma linguagem matemática precisa para descrever problemas de decisão sequencial quanto algoritmos concretos para resolvê-los. Quando conhecemos completamente o ambiente, ou seja, sabemos exatamente quais são as probabilidades de transição e as recompensas para cada ação em cada estado, podemos usar técnicas como programação dinâmica para encontrar a solução ótima de forma determinística.

Quando não conhecemos essas informações previamente, podemos usar algoritmos de aprendizado por reforço que permitem que um agente aprenda a política ótima através da experiência, tentando diferentes ações e observando os resultados.

O MDP serve como base teórica para praticamente toda a área de aprendizado por reforço, que tem sido fundamental para avanços recentes em inteligência artificial, incluindo sistemas que conseguem jogar videogames melhor que humanos, controlar sistemas de recomendação em plataformas digitais e otimizar o consumo de energia em datacenters.

A elegância matemática do MDP, combinada com sua aplicabilidade prática, torna-se uma ferramenta indispensável para qualquer pessoa interessada em resolver problemas de decisão complexa de forma sistemática e eficiente, demonstrando como conceitos matemáticos abstratos podem ter impacto direto e tangível na resolução de problemas do mundo real.

8.2. Exemplo de um Problema Markoviano

Podemos imaginar um robô que se desloca continuamente entre três salas distintas: A, B e C. O movimento desse robô não é completamente livre, mas sim governado por probabilidades que descrevem as transições possíveis entre os ambientes.

Na sala A, o robô apresenta um comportamento de permanência bastante significativo, existe 70% de chance de ele permanecer na mesma sala, sem realizar nenhuma transição. No entanto, há também uma probabilidade de 30% de se deslocar para a sala B, seguindo o caminho permitido pelas portas.

Quando o robô está na sala B, as opções de movimento são mais distribuídas. Ele possui 40% de probabilidade de voltar para a sala A, retornando ao ponto inicial, e 60% de probabilidade de avançar para a sala C, ampliando seu percurso no ambiente.

Por fim, ao chegar à sala C, o robô não possui alternativas, existe uma probabilidade de 100% de voltar para a sala A. Isso significa que a sala C funciona como um ponto de retorno obrigatório, reiniciando o ciclo e garantindo que o robô sempre volte a passar pelo estado inicial.

Esse cenário exemplifica de forma clara uma Cadeia de Markov, na qual o próximo estado do sistema depende apenas do estado atual e não do histórico anterior de movimentos. Aqui, as transições entre salas são totalmente definidas pelas probabilidades associadas a cada porta, permitindo descrever matematicamente o comportamento do robô ao longo do tempo.

8.3. Cadeia de Markov do Problema Proposto

O grafo mostrado na Figura-8.1 representa de forma visual a Cadeia de Markov que descrevemos para o robô que se movimenta entre três salas: A, B e C. Cada círculo corresponde a um estado (uma sala) e as setas indicam as transições possíveis entre esses estados, acompanhadas das probabilidades associadas.

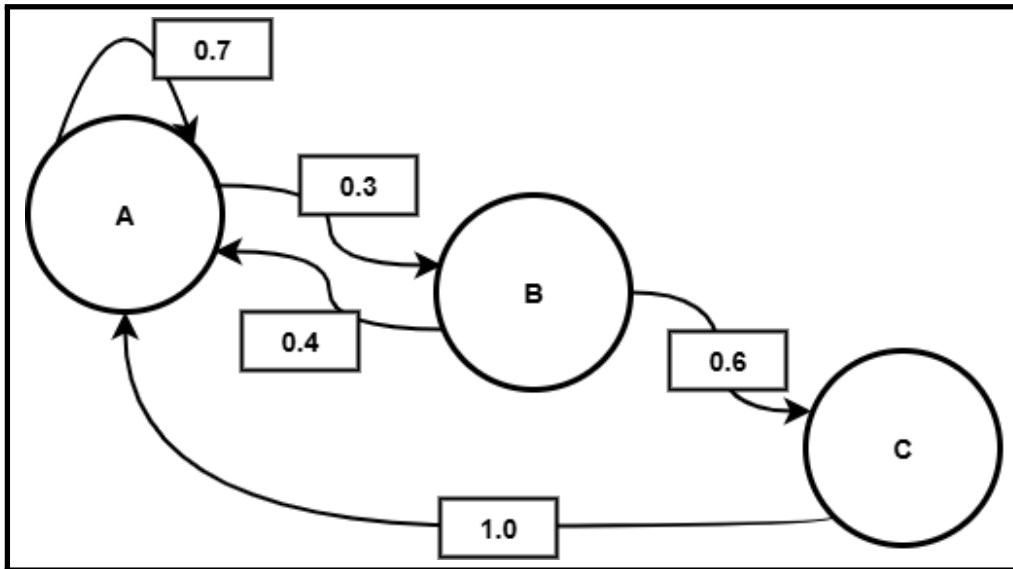


Figura 8.1: Exemplo de uma Cadeia de Markov

- Sala A: possui duas possibilidades.
 - Com probabilidade de 0,7 (70%), o robô permanece em A (o self-loop representado pela seta que sai e volta para A).
 - Com probabilidade de 0,3 (30%), ele se desloca para a sala B.
- Sala B: apresenta dois caminhos de saída.
 - O robô tem 40% de chance (0,4) de retornar para A.
 - E 60% de chance (0,6) de seguir para a sala C.
- Sala C: é um estado que funciona como um ponto de retorno obrigatório.
 - A seta mostra que, com probabilidade de 1,0 (100%)
 - A seta mostra que, com probabilidade de 1,0 (100%), o robô volta sempre para a sala A, reiniciando o ciclo.

Esse grafo deixa claro que o sistema segue a propriedade de Markov, ou seja, o próximo movimento do robô depende apenas da sala atual em que ele se encontra e das probabilidades associadas às transições possíveis.

Na prática, isso quer dizer que, ao observar o robô em determinado momento, conseguimos descrever completamente suas chances de movimento futuro sem precisar considerar o histórico dos passos anteriores. Esse é o núcleo do funcionamento de uma Cadeia de Markov, representado de forma intuitiva pelas setas e probabilidades do grafo.

8.4. Matriz de Transição do Exemplo Proposto

Considerando as transições do grafo, podemos descrever o comportamento do robô entre as três salas de forma probabilística. Quando o robô está na sala *A*, ele possui 70% de chance de permanecer na mesma sala, 30% de chance de se deslocar para a sala *B* e nenhuma chance de ir diretamente para a sala *C*.

Já na sala *B*, o robô apresenta 40% de chance de retornar para a sala *A*, não tem possibilidade de permanecer em *B* e tem 60% de chance de seguir para a sala *C*.

Por fim, ao estar na sala *C*, a transição é determinística: o robô retorna para a sala *A* com probabilidade de 100%, sem qualquer chance de ir para *B* ou permanecer em *C*. Esse comportamento pode ser formalmente representado pela seguinte **matriz de transição**:

$$P = \begin{bmatrix} 0.7 & 0.3 & 0.0 \\ 0.4 & 0.0 & 0.6 \\ 1.0 & 0.0 & 0.0 \end{bmatrix}$$

Nessa matriz, cada linha corresponde a um estado atual (as salas) e cada coluna indica o próximo estado possível, de modo que os valores representam as probabilidades de transição. A estrutura garante que a soma de cada linha seja igual a 1, refletindo todas as possibilidades de movimento a partir de um estado.

- A interpretação da matriz de transição pode ser descrita da seguinte forma:
 - A primeira linha $[0.7, 0.3, 0.0]$ mostra que, quando o robô está na sala *A*, ele tem 70% de probabilidade de permanecer na mesma sala, 30% de probabilidade de se deslocar para a sala *B* e nenhuma possibilidade de ir diretamente para a sala *C*.
 - A segunda linha $[0.4, 0.0, 0.6]$ indica que, estando na sala *B*, o robô tem 40% de chance de retornar para a sala *A*, 0% de permanecer em *B* e 60% de probabilidade de seguir para a sala *C*.
 - Já a terceira linha $[1.0, 0.0, 0.0]$ mostra que, ao estar na sala *C*, o robô sempre retorna para a sala *A*, sem qualquer chance de permanecer em *C* ou de se deslocar para a sala *B*.

Esse conjunto de informações evidencia o comportamento probabilístico do sistema e como as transições entre os estados estão rigidamente definidas pelas probabilidades atribuídas a cada linha da matriz.

Essa matriz de transição pode ser utilizada de diferentes formas dentro do estudo de cadeias de Markov. Em primeiro lugar, ela permite prever o estado futuro do sistema. Para isso, basta multiplicar a distribuição de probabilidades inicial do agente pela matriz *P*, o que resulta nas probabilidades de estar em cada estado após um certo número de passos.

Além disso, a matriz possibilita analisar o comportamento de longo prazo, já que, por meio de sucessivas multiplicações, podemos calcular a chamada distribuição estacionária. Essa distribuição indica as probabilidades de o agente permanecer em cada estado quando o número de transições tende ao infinito, fornecendo uma visão de equilíbrio do sistema.

Por fim, a matriz também pode ser aplicada em simulações, onde múltiplas iterações são realizadas para observar o comportamento dinâmico do processo ao longo do tempo, permitindo identificar padrões, frequências de visita a cada estado e probabilidades acumuladas de trajetórias específicas.

Entendendo a Equação de Bellman

9.1. Introdução

A Equação de Bellman é uma das expressões matemáticas mais sofisticadas e efetivas na teoria da otimização dinâmica, servindo como a base teórica para a programação dinâmica e, por conseguinte, para uma significativa parcela do aprendizado por reforço contemporâneo. Criada pelo matemático norte-americano Richard Ernest Bellman nos anos 1950, essa equação sintetiza de maneira clara e precisa o Princípio da Otimalidade, convertendo percepções teóricas em um instrumento matemático rigoroso e passível de tratamento computacional.

A relevância da Equação de Bellman vai além de suas aplicações diretas na área da ciência da computação, abrangendo domínios variados, como economia, engenharia de controle, biologia matemática e teoria dos jogos. Sua elaboração matemática refinada possibilita a decomposição de problemas complexos de otimização, que requerem decisões sequenciais, em uma sucessão de subproblemas mais simples, os quais podem ser resolvidos de maneira ótima. Essa decomposição possibilita não apenas a resolução de problemas que seriam computacionalmente inviáveis por meio de métodos tradicionais, mas também oferece percepções significativas sobre a estrutura subjacente dos processos de tomada de decisão ótima.

A equação impõe uma relação recursiva essencial entre o valor ótimo de um determinado estado e os valores ótimos dos estados que podem ser acessados a partir desse estado. Essa relação recursiva expressa a essência do raciocínio dinâmico, ou seja, para realizar a melhor escolha em qualquer instante, é necessário levar em conta não somente as consequências imediatas das ações, mas também a maneira como essas ações influenciam nossas oportunidades futuras de otimização. A beleza matemática da equação encontra-se na sua habilidade de traduzir essa intuição complexa por meio de uma formulação matemática rigorosa e eficiente do ponto de vista computacional.

Atualmente, a Equação de Bellman adquiriu uma importância ainda mais significativa como base teórica do aprendizado por reforço. Algoritmos contemporâneos, como Q-learning, SARSA e técnicas de gradiente de política, derivam diretamente dos princípios contidos na equação, evidenciando sua relevância contínua em questões de vanguarda tecnológica. A habilidade da equação em gerenciar incertezas, recompensas e espaços de estado complexos a torna essencial para o avanço de sistemas inteligentes que necessitam aprender e se adaptar em ambientes imprevisíveis.

9.2. Ambientes Determinísticos e Estocásticos

Um sistema determinístico é aquele no qual conseguimos prever exatamente o que vai acontecer, sem margem para incertezas. Já um sistema estocástico descreve o que provavelmente ocorrerá, incorporando a ideia de incerteza expressa em probabilidade nos resultados.

A principal vantagem do sistema estocástico sobre o determinístico é que ele não exige o conhecimento completo de todas as variáveis que influenciam o processo. Em vez disso, é possível construir uma função baseada em dados, de forma que as diferentes situações possíveis sejam representadas dentro de uma estrutura estatística previamente modelada.

No caso de um ambiente estocástico, a característica central é que não conseguimos prever o resultado apenas a partir do estado atual do sistema. A incerteza é inerente e pode ser significativa.

Um exemplo clássico é o lançamento de dados. Se não conhecemos em detalhe as condições de lançamento e de como a força aplicada, o ângulo ou a resistência do ar, não seremos capazes de determinar de antemão o valor que sairá. Nesse cenário, a única forma de descrever o fenômeno é por meio da probabilidade de cada face aparecer.

Assim, enquanto sistemas determinísticos fornecem previsões exatas, sistemas estocásticos permitem lidar com a incerteza de maneira estruturada, oferecendo um modelo mais realista para fenômenos complexos em que nem todas as variáveis são conhecidas ou mensuráveis.

9.3. Parâmetros que Influenciam a Equação de Bellman

A taxa de aprendizado e o fator de desconto representam dois parâmetros fundamentais que exercem influência determinante sobre o comportamento e a convergência da equação de Bellman em algoritmos de aprendizado por reforço, moldando diretamente a forma como um agente aprende a tomar decisões ótimas em ambientes complexos e dinâmicos. Estes parâmetros atuam como mecanismos de controle que regulam tanto a velocidade quanto a qualidade do processo de aprendizado, determinando se o algoritmo será capaz de encontrar as políticas ótimas de forma eficiente e estável.

A taxa de aprendizado, frequentemente denotada pela letra grega α , controla a magnitude das atualizações realizadas nos valores estimados a cada iteração da equação de Bellman, funcionando essencialmente como um regulador da velocidade de aprendizado do sistema. Quando a taxa de aprendizado assume valores elevados, próximos a 1.0, o algoritmo realiza atualizações agressivas que incorporam rapidamente novas informações, permitindo adaptação veloz a mudanças no ambiente, mas ao custo de maior instabilidade e suscetibilidade a ruído. Porém, quando usadas taxas de aprendizado baixas, próximas a zero, resultam em atualizações conservadoras que conferem maior estabilidade ao processo, porém podem levar a uma convergência excessivamente lenta, especialmente em ambientes que exigem adaptação rápida ou quando há limitações temporais para o treinamento.

O fator de desconto, tradicionalmente representado pela letra grega γ , exerce uma influência ainda mais fundamental sobre a natureza das soluções encontradas pela equação de Bellman, determinando o horizonte temporal considerado pelo agente em suas decisões e a importância relativa atribuída a recompensas futuras versus imediatas. Este parâmetro varia tipicamente entre 0 e 1, onde valores próximos a zero fazem com que o agente priorize exclusivamente recompensas imediatas, desenvolvendo uma perspectiva míope que pode ser adequada para tarefas episódicas curtas, mas inadequada para problemas que exigem planejamento de longo prazo.

Quando o fator de desconto se aproxima de 1.0, o agente desenvolve uma perspectiva de longo prazo que valoriza igualmente recompensas futuras e presentes, permitindo o desenvolvimento de estratégias sofisticadas que podem sacrificar ganhos imediatos em favor de benefícios substanciais no futuro. Esta configuração é particularmente importante em problemas de planejamento estratégico, jogos complexos ou sistemas de controle onde decisões presentes têm consequências duradouras. No entanto, fatores de desconto muito próximos a 1.0 podem tornar o problema computacionalmente mais desafiador e potencialmente instável.

A interação entre taxa de aprendizado e fator de desconto na equação de Bellman cria dinâmicas complexas que afetam profundamente o comportamento convergente do algoritmo. Em cenários onde o fator de desconto é elevado, indicando valorização de recompensas futuras, uma taxa de aprendizado inadequadamente alta pode causar instabilidade severa, pois pequenos erros nas estimativas de valores futuros são amplificados através das iterações sucessivas da equação. Quando o fator de desconto é baixo, o sistema tende a ser mais robusto a taxas de aprendizado elevadas, pois o impacto de erros de estimação se dissipa rapidamente devido ao menor peso atribuído a estados futuros.

A escolha conjunta destes parâmetros também influencia significativamente a capacidade do algoritmo de lidar com diferentes tipos de ambientes e problemas. Em ambientes estacionários com recompensas bem definidas, combinações conservadoras de taxa de aprendizado moderadamente baixa com fator de desconto apropriado ao horizonte do problema tendem a produzir resultados estáveis e confiáveis. Por outro lado, em ambientes não estacionários ou com alta variabilidade, pode ser necessário adotar taxas de aprendizado mais elevadas para permitir adaptação contínua, mesmo que isso implique em alguma perda de estabilidade.

O impacto destes parâmetros na equação de Bellman estende-se também à capacidade de generalização e robustez das políticas aprendidas. Configurações que favorecem convergência rápida nem sempre resultam em políticas que generalizam bem para situações não vistas durante o treinamento, enquanto configurações mais conservadoras podem produzir políticas mais robustas, porém, ao custo de maior tempo de treinamento. Esta tensão entre velocidade de convergência e qualidade da solução final representa um dos principais desafios na calibração adequada destes parâmetros em aplicações práticas de aprendizado por reforço.

9.4. Descrevendo a Equação de Bellman

A equação de Bellman apresentada na Figura-9.1 representa o algoritmo Q-learning, uma das implementações mais fundamentais e amplamente utilizadas do aprendizado por reforço, que atualiza iterativamente os valores Q (qualidade das ações) para permitir que um agente aprenda a política ótima através da experiência direta com o ambiente. Esta formulação específica combina informações sobre recompensas imediatas e futuras para refinar continuamente as estimativas de valor das ações em cada estado.

$$\begin{array}{c}
 \underbrace{Q'(s, a)}_{\text{Novo Valor Q}} = \underbrace{Q(s, a)}_{\text{Valor Atual Q}} + \underbrace{\alpha}_{\text{Taxa de Aprendizado}} \left[\underbrace{R(s, a)}_{\substack{\text{Recompensa} \\ \text{pela Ação} \\ \text{Atual}}} + \underbrace{\gamma}_{\text{Taxa de Desconto}} \underbrace{\max Q'(s', a')}_{\substack{\text{O Maior Valor} \\ \text{de Q entre as} \\ \text{próximas ações possíveis}}} - \underbrace{Q(s, a)}_{\text{Valor Q Atual}} \right] \quad (9.1)
 \end{array}$$

Recompensa Futura

O funcionamento da equação inicia-se com a definição do novo valor Q ($Q'(s, a)$) localizado no lado esquerdo, representando a estimativa atualizada da qualidade de executar a ação 'a' no estado 's'. Esta atualização é calculada tomando como base o valor atual $Q(s, a)$ e aplicando uma correção baseada na diferença entre a experiência real observada e a estimativa prévia. O valor atual $Q(s, a)$ serve como ponto de partida, incorporando todo o conhecimento acumulado até o momento sobre a qualidade desta combinação estado-ação específica.

A taxa de aprendizado (α) funciona como um regulador crucial que determina quanto da nova informação será incorporada à estimativa existente. Quando α assume valores próximos a 1, o algoritmo dá maior peso às experiências recentes, permitindo adaptação rápida, mas potencialmente instável. Valores baixos de α tornam o aprendizado mais conservador e estável, porém mais lento para se adaptar a mudanças no ambiente. Este parâmetro multiplica toda a componente de correção, determinando a magnitude da atualização realizada a cada iteração.

A componente de recompensa imediata $R(s, a)$ representa o feedback direto recebido do ambiente após executar a ação 'a' no estado 's'. Esta recompensa imediata constitui a informação mais confiável disponível, pois foi diretamente observada, ou seja, não estimada. Ela forma a base sobre a qual as atualizações são construídas, fornecendo o sinal de aprendizado primário que orienta o algoritmo em direção a comportamentos mais recompensadores.

O fator de desconto (γ) modula a importância atribuída às recompensas futuras, controlando o horizonte temporal considerado pelo agente em suas decisões. Valores de γ próximos a 1 fazem com que o agente valorize igualmente recompensas presentes e futuras, desenvolvendo estratégias de longo prazo, enquanto valores próximos a 0 criam um comportamento mais imediato, focado em ganhos imediatos. Este parâmetro multiplica a estimativa do valor futuro máximo, determinando quanto peso é dado às consequências de longo prazo das ações atuais.

A expressão $\max Q(s', a')$ representa a estimativa do valor máximo esperado no próximo estado s' , considerando todas as ações possíveis a' disponíveis naquele estado. Esta componente incorpora a informação sobre o futuro potencial, assumindo que o agente agirá otimamente a partir do próximo estado. O operador max reflete o princípio de optimalidade de Bellman, assumindo que decisões futuras serão tomadas de forma a maximizar o valor esperado.

O termo entre parênteses $[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a)]$ constitui o erro de predição temporal, que quantifica a discrepância entre a nova estimativa baseada na experiência real e a estimativa anterior. Quando este erro é positivo, indica que a experiência foi melhor do que o esperado, resultando em aumento do valor Q . Quando negativo, sugere que a experiência foi pior que a expectativa, levando a uma redução do valor estimado.

A estrutura multiplicativa $\alpha \times [\text{erro temporal}]$ garante que as atualizações sejam proporcionais tanto à magnitude do erro quanto à taxa de aprendizado configurada. Esta formulação permite que o algoritmo faça correções graduais, mas consistentes, aproximando-se progressivamente dos valores Q ótimos através de experiências repetidas. A convergência ocorre quando os valores Q se estabilizam, refletindo as expectativas corretas sobre as recompensas futuras para cada combinação estado-ação.

O processo iterativo implícito nesta equação permite que o agente melhore continuamente suas estimativas sem necessidade de conhecimento prévio sobre o ambiente. A cada experiência (s, a, r, s') , o algoritmo atualiza $Q(s,a)$ incorporando a nova informação observada, refinando gradualmente sua compreensão sobre quais ações são mais valiosas em cada situação. Esta capacidade de aprender através da experiência direta, sem modelo explícito do ambiente, torna o Q-learning particularmente valioso em domínios complexos onde modelagem analítica é impraticável.

9.5. Equação de Bellman em Ambientes Determinísticos

Quando aplicamos a equação de Bellman em um ambiente determinístico, ocorre uma simplificação importante. De forma geral, a equação foi criada para ambientes estocásticos, nos quais uma mesma ação pode levar a diferentes estados futuros com probabilidades distintas. Nesse caso, o cálculo do valor futuro esperado envolve uma soma ponderada de recompensas e valores de estados, considerando as probabilidades de cada transição.

Já em um ambiente determinístico, não existe incerteza: dado um estado e uma ação, o próximo estado e a recompensa são sempre os mesmos. Isso significa que a probabilidade associada ao próximo estado é 1, enquanto todas as outras são 0. Assim, a equação de Bellman se reduz a uma forma mais direta, típica do Q-Learning clássico:

Nessa configuração, o agente atualiza sua Q-table diretamente a partir do resultado fixo de cada interação, sem precisar estimar expectativas sobre múltiplos cenários possíveis. Como consequência, o aprendizado se torna mais simples e tende a convergir rapidamente para a política ótima.

Na prática, ambientes determinísticos apresentam diferenças claras em relação aos estocásticos. As transições são totalmente previsíveis, pois cada ação leva sempre ao mesmo próximo estado. As recompensas também são fixas, de modo que um par estado-ação gera sempre o mesmo retorno, o que facilita o processo de atualização da Q-table.

Isso faz com que a convergência seja mais rápida, já que não há ruído ou variabilidade a serem considerados. Além disso, há menor necessidade de exploração, pois, em um ambiente estático, basta identificar o melhor caminho uma única vez para que ele seja seguido de forma consistente. No entanto, essa configuração apresenta menor realismo, uma vez que a ausência de estocasticidade limita a aplicabilidade a problemas reais, que normalmente envolvem incertezas e mudanças ao longo do tempo.

Dessa forma, podemos aplicar a equação de Bellman de acordo com o tipo de ambiente, determinístico ou estocástico. Considere, então, a equação de Bellman para um ambiente determinístico:

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

Nela, $V(s)$ é o valor do estado atual, enquanto $V(s')$ representa o valor para se alcançar o próximo estado após executarmos a ação a . O termo $R(s, a)$ corresponde à recompensa recebida pelo agente após realizar a ação a no estado s . Como o agente pode realizar diferentes ações, utiliza-se o valor máximo porque o objetivo é sempre buscar o estado mais vantajoso. Por fim, γ é o fator de desconto, que define a importância atribuída às recompensas futuras em relação às imediatas.

Assim, embora o ambiente determinístico simplifique o uso da equação de Bellman e acelere a convergência, ele não reflete a complexidade de cenários reais, servindo mais como um ponto de partida didático e como teste para validação de algoritmos de aprendizado por reforço.

9.6. Restrições e Obstáculos

Não obstante a sua elegância teórica e extensa aplicabilidade, a Equação de Bellman apresenta limitações relevantes que restringem sua utilização prática em determinados contextos. Compreender essas limitações é fundamental para a aplicação eficaz da equação e para o desenvolvimento de extensões que tratem de suas deficiências.

A maldição da dimensionalidade¹ talvez constitua o principal obstáculo na aplicação da Equação de Bellman. Conforme a quantidade de variáveis de estado se eleva, o espaço de estados aumenta de forma exponencial, o que torna a representação tabular da função de valor inviável do ponto de vista computacional. Em um problema que envolve d variáveis de estado, onde cada variável pode assumir n valores discretos, o tamanho do espaço de estados é igual a n elevado à potência d .

Essa limitação dimensional evidencia-se tanto nas exigências de memória quanto no tempo de processamento. As abordagens para minimizar este problema podem incluir o uso de aproximação de funções, consolidação de estados e decomposição em níveis hierárquicos; no entanto, cada uma delas implica em trocas entre precisão e eficiência computacional.

A convergência da Equação de Bellman, apesar de estar teoricamente assegurada em condições adequadas, pode ocorrer de maneira extremamente lenta. A taxa de convergência é restringida pelo fator de desconto γ , sendo que valores próximos a 1 ocasionam uma convergência progressivamente lenta. Em situações nas quais as recompensas futuras têm relevância significativa (com γ aproximando-se de 1), o total de iterações requeridas para alcançar a convergência pode ser excessivamente elevado.

Questões relacionadas à estabilidade numérica podem surgir em implementações práticas, especialmente quando se faz uso da aproximação de funções. A junção de aproximação de função, bootstrapping (emprego de estimativas para atualizar outras estimativas) e aprendizado off-policy pode resultar em divergência, apesar de cada componente isolado apresentar estabilidade. O fenômeno denominado "triângulo mortal" no aprendizado por reforço demonstra de que maneira a interação desses componentes pode ocasionar instabilidade.

A definição apropriada da função de recompensa constitui um desafio considerável. A Equação de Bellman parte da premissa de que os objetivos podem ser representados por meio de uma função de recompensa escalar; no entanto, diversas questões do mundo real envolvem objetivos que são múltiplos, conflitantes ou de difícil quantificação.

Assim, complexas em funções de recompensa frequentemente necessitam de considerável especialização no domínio e podem acarretar vieses não intencionais. A característica de Markov, essencial para a legitimidade da Equação de Bellman, pode ser infringida em casos práticos nos quais informações pertinentes não estão inteiramente encapsuladas no estado presente. Dificuldades relacionadas à observabilidade parcial, à capacidade de memória restrita ou a dependências temporais intrincadas podem comprometer a validade da suposição markoviana, restringindo a aplicação direta da equação.

¹A maldição da dimensionalidade descreve os problemas que surgem em espaços de alta dimensão, onde os dados se tornam esparsos e as distâncias perdem significado, dificultando a análise e aumentando o risco de overfitting.

As questões relativas à exploração versus exploração (exploration vs. aproveitamento⁶) introduzem complexidades adicionais em cenários de aprendizado. A Equação de Bellman pressupõe um entendimento abrangente da dinâmica do ambiente. Entretanto, em contextos de aprendizado, os agentes precisam conciliar a exploração de ações ainda não conhecidas com a utilização do conhecimento já adquirido. Esse compromisso não é diretamente contemplado pela equação e demanda considerações algorítmicas suplementares.

10

Q-Learning sem Exploração do Ambiente

10.1. Introdução

O Q-Learning é um algoritmo de aprendizado por reforço off-policy¹ que busca estimar a função de valor $Q(s, a)$ para cada par estado-ação, determinando o retorno esperado ao seguir a política ótima. Normalmente, a tomada de decisão é feita por meio de uma estratégia de equilíbrio entre exploração (testar ações novas para descobrir alternativas melhores) e aproveitamento (repetir as ações já conhecidas como boas), sendo o método ϵ -greedy o mais comum para este fim.

Quando se remove a fase de exploração, por exemplo, $\epsilon = 0$ desde o início, o algoritmo se torna puramente exploratório no sentido de reforçar apenas as ações já conhecidas, sem nunca testar novas opções. Nesse caso, a escolha de ação sempre é feita pelo argmax dos valores atuais da Q-table, sem introduzir aleatoriedade.

¹o agente aprende os valores de uma política ótima independentemente da política que está sendo seguida para coletar dados. Exemplo clássico: Q-Learning.

10.2. Características do Algoritmo Q-Learning sem Exploração

Quando o Q-Learning é configurado sem fase de exploração, ele apresenta um conjunto específico de características que influenciam diretamente a forma como o agente aprende e interage com o ambiente. Nesse cenário, ocorre uma convergência restrita, pois o aprendizado pode chegar rapidamente a um ponto de estabilidade, mas muitas vezes esse ponto representa uma política subótima. Isso acontece porque, ao não explorar ações desconhecidas, o agente deixa de identificar alternativas que poderiam gerar maiores recompensas.

Outro aspecto importante é a sensibilidade ao estado inicial e aos valores iniciais da Q-table. Caso os valores atribuídos inicialmente favoreçam uma determinada ação, o agente tende a repeti-la indefinidamente, mesmo que existam outras opções mais vantajosas. Esse viés inicial pode comprometer a qualidade da política aprendida.

Essa abordagem encontra aplicabilidade principalmente em ambientes totalmente conhecidos, determinísticos e estáticos, onde a incerteza é mínima e a política ótima já é parcialmente conhecida. Nessas condições, a ausência de exploração não impede o bom desempenho, pois o espaço de busca já está mapeado ou próximo de estar.

No entanto, existe um risco considerável em ambientes dinâmicos. Se o ambiente sofre alterações ou possui variabilidade natural, a falta de exploração impede que o agente se adapte às novas condições, resultando em degradação de desempenho e possivelmente em decisões ineficientes.

10.3. A Equação de Bellman sem Exploração

O Q-Learning é um algoritmo de aprendizado por reforço do tipo off-policy que busca estimar a função de valor $Q(s, a)$ para cada par estado—ação, determinando o retorno esperado ao seguir a política ótima. A atualização dos valores da Q-table é realizada segundo a equação de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right],$$

onde α é a taxa de aprendizado, γ é o fator de desconto, r é a recompensa imediata e $\max_{a'} Q(s', a')$ representa a estimativa do melhor valor futuro esperado. Normalmente, a escolha da ação combina exploração (testar ações novas) e exploração (repetir ações conhecidas como boas), sendo o método ε -greedy o mais comum.

Quando a exploração é removida — por exemplo, configurando $\varepsilon = 0$ desde o início —, o agente sempre seleciona a ação com maior valor na Q-table, sem introduzir aleatoriedade. Isso significa que o termo $\max_{a'} Q(s', a')$ é calculado apenas com base nas ações já conhecidas, limitando a coleta de novas experiências. Essa configuração pode levar a uma convergência restrita, pois o aprendizado tende a estabilizar rapidamente, mas para uma política potencialmente subótima, já que o agente não busca ações desconhecidas que possam gerar recompensas maiores.

Além disso, há uma sensibilidade ao estado inicial e aos valores iniciais da Q-table: se no início uma ação recebe um valor ligeiramente maior (por acaso ou inicialização), ela será reforçada repetidamente, enquanto outras não terão chance de serem testadas. Isso é especialmente crítico em ambientes estocásticos ou dinâmicos, nos quais a ausência de exploração impede que a política se adapte a mudanças, levando a quedas de desempenho. Por outro lado, essa abordagem pode ser útil em ambientes determinísticos, totalmente conhecidos e estáticos, nos quais a política ótima já é parcialmente conhecida e a prioridade é a rapidez de convergência.

10.4. Funcionamento do Algoritmo Q-Learning sem Exploração

A Figura-10.1 descreve o funcionamento do algoritmo Q-Learning sem a fase de exploração. O processo inicia com a criação da Q-table, atribuindo valores iniciais (normalmente zero) para todos os pares estado-ação. Em seguida, o agente observa o estado atual do ambiente e escolhe, de forma determinística, a ação cujo valor Q seja o maior naquele estado, adotando assim uma política puramente greedy.

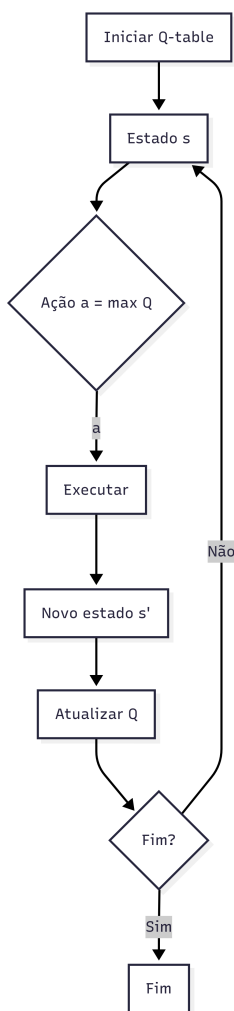


Figura 10.1: Funcionamento Geral do QLearning Sem Exploração

Não há tentativa de testar ações alternativas, de modo que a decisão é sempre guiada pelos valores já conhecidos. A ação escolhida é executada, o agente recebe a recompensa e observa o novo estado resultante, usando essas informações para atualizar a Q-table conforme a equação de Bellman, reforçando ainda mais a escolha anterior. Esse ciclo de observação, decisão e atualização prossegue até o término do episódio, quando o processo recomeça em um novo episódio ou é finalizado.

Por não haver exploração, essa abordagem tende a convergir rapidamente, mas corre o risco de encontrar apenas soluções subótimas, pois não há busca por opções desconhecidas que possam gerar maior retorno. Em ambientes determinísticos e totalmente conhecidos, ela pode ser eficiente; porém, em contextos dinâmicos ou incertos, sua incapacidade de se adaptar a mudanças pode levar a queda de desempenho e falta de generalização.

10.5. Vantagens do Q-Learning sem Exploração

A principal vantagem é a velocidade de convergência. Como o agente sempre escolhe a ação de maior valor conhecido, ele reforça rapidamente as decisões iniciais, consolidando uma política estável em menos tempo.

Essa característica o torna útil em ambientes totalmente conhecidos, determinísticos e estáticos, onde a política ótima já é parcialmente conhecida ou pode ser deduzida sem a necessidade de explorar opções desconhecidas.

Outro ponto positivo é a simplicidade de implementação, pois ao remover a exploração, elimina-se a necessidade de mecanismos como o ϵ -greedy, reduzindo a variabilidade e o custo computacional de testes aleatórios.

10.6. Limitações do Q-Learning sem Exploração

Por outro lado, essa configuração tem alta propensão a convergir para soluções subótimas, pois nunca testa alternativas que poderiam resultar em recompensas maiores.

A ausência de exploração torna o algoritmo sensível às condições iniciais, já que valores de Q inicializados de forma enviesada podem prender o agente em comportamentos repetitivos.

Em ambientes dinâmicos ou incertos, o desempenho pode cair rapidamente, pois não há capacidade de adaptação a mudanças nas recompensas ou transições. Essa falta de flexibilidade significa que, mesmo que o ambiente apresente novas oportunidades, o agente permanecerá preso à política previamente aprendida, sem jamais atualizá-la com base em novas possibilidades.

11

Q-Learning com Exploração do Ambiente

11.1. Introdução

O Q-Learning é um dos algoritmos mais essenciais e frequentemente aplicados em aprendizado por reforço, sendo categorizado como um método de diferença temporal sem modelo. Criado por Chris Watkins em 1989, esse algoritmo possibilita que um agente aprenda a realizar escolhas ideais em um ambiente por meio da interação direta, dispensando a necessidade de um conhecimento prévio acerca da dinâmica desse ambiente.

O princípio fundamental do Q-Learning está associado à função Q , identificada também como função de valor-ação, a qual calcula o valor esperado ao executar uma ação particular em um estado específico e subsequente à adoção da política ótima. A função, identificada como $Q(s,a)$, expressa a recompensa total esperada ao realizar a ação 'a' no estado 's' e, em seguida, adotar a estratégia mais eficiente disponível. A finalidade do algoritmo consiste em apreender essa função Q de maneira iterativa, possibilitando ao agente discernir qual ação adotar em cada circunstância, a fim de otimizar suas recompensas no longo prazo.

A função Q , comumente chamada de função valor-ação, a qual é expressa por meio de uma estrutura denominada tabela Q . Esta tabela a seguir associa cada par estado-ação a um valor $Q(s,a)$, e representa a expectativa da recompensa total a ser recebida ao realizar a ação a no estado s , e posteriormente seguir a política ótima a partir desse momento.

11.2. Funcionamento do Algoritmo Q-Learning

A operação do Q-learning baseia-se na criação de uma tabela denominada Q-table, que armazena os valores Q , os quais representam a "qualidade"(ou utilidade) de executar uma ação específica em um estado determinado. De maneira formal, o valor Q , expresso como $Q(s,a)$, refere-se à expectativa de recompensa resultante da escolha da ação a no estado s , levando em consideração a implementação de

uma política ótima posteriormente. A tabela é estruturada com valores aleatórios e é progressivamente atualizada, levando em conta as interações do agente com o meio.

O algoritmo segue os seguintes passos:

1. A Q-table é iniciada, normalmente com valores iguais a zero ou com números aleatórios, para cada par estado-ação (s, a) . Parâmetros tais como a taxa de aprendizado α , o fator de desconto γ e a estratégia de exploração (por exemplo, ϵ -greedy) são estabelecidos.
2. Interação com o ambiente: A cada iteração (ou episódio), o agente analisa o estado presente s , seleciona uma ação a fundamentada em uma política, como a ϵ -greedy, que equilibra a exploração (opções aleatórias) e a exploração do conhecimento adquirido (opção pela ação que apresenta o maior valor Q). Após a execução da ação, o agente recebe uma recompensa r e observa o estado seguinte s' .
3. Atualização da Q-table: O valor $Q(s, a)$ é atualizado usando a equação de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Aqui, α (taxa de aprendizado) determina o peso dado às novas informações, γ (fator de desconto, entre 0 e 1) pondera a importância de recompensas futuras, r é a recompensa imediata, e $\max_{a'} Q(s', a')$ representa o maior valor Q possível no próximo estado s' .

4. Repetição: O processo se repete por vários episódios ou até que a Q-table convirja, ou seja, os valores Q estabilizem, indicando que o agente aprendeu uma política próxima da ótima.

A estratégia ϵ -greedy é frequentemente utilizada na escolha das ações: com uma probabilidade de ϵ , o agente explora uma ação selecionada aleatoriamente; enquanto, com uma probabilidade de $1 - \epsilon$, ele opta pela ação que demonstra o maior valor Q. Com o avanço do tempo, ϵ pode ser diminuído para apoiar a exploração do conhecimento adquirido.

O Q-learning demonstra eficiência ao se aproximar da política ideal em ambientes markovianos¹, nos quais o estado seguinte é exclusivamente influenciado pelo estado atual e pela ação realizada, mesmo sem a presença de um modelo do ambiente. Entretanto, observa-se uma limitação: em contextos onde existem extensos intervalos de estados ou ações, a Q-table pode se tornar impraticável, demandando a utilização de abordagens como o Deep Q-Learning, que faz uso de redes neurais para a aproximação da função Q . Além disso, o algoritmo pode demonstrar sensibilidade em relação à escolha dos hiperparâmetros (α , γ , ϵ) e à eficácia da exploração efetuada pelo agente.

¹Um ambiente é markoviano quando não precisamos da história inteira para prever o próximo passo; basta conhecer o estado atual e a ação escolhida

11.3. Exploração versus Aproveitamento

A harmonia entre a exploração e a aproveitamento representa um dos desafios mais cruciais no aprendizado por reforço, particularmente no Q-Learning, configurando um dilema que cada agente inteligente precisa enfrentar para maximizar seu desempenho ao longo do tempo. Durante o processo de aprendizado, o agente enfrentará, de modo constante, uma escolha essencial: deverá utilizar o conhecimento já assimilado para assegurar recompensas certas e familiares, ou deverá ousar ao explorar territórios desconhecidos, na esperança de descobrir estratégias mais eficazes. A tensão existente entre segurança e descoberta é reconhecida como o trade-off entre exploração e aproveitamento, sendo que a sua resolução apropriada impacta de maneira significativa o sucesso do algoritmo.

A exploração refere-se à prática de tentar realizar ações que, a princípio, podem não aparentar ser as mais adequadas, mas têm a capacidade de revelar informações relevantes sobre o contexto e, por conseguinte, levar a recompensas mais significativas no futuro. Ao investigar, um agente, essencialmente, abdica de benefícios imediatos em favor da aquisição de conhecimento que pode ser benéfico a longo prazo. Esse procedimento é essencial, especialmente nas etapas iniciais do aprendizado, nas quais o agente possui informações limitadas sobre as repercussões de suas ações e precisa desenvolver um mapeamento abrangente das condições e ações disponíveis.

O aproveitamento refere-se à utilização do conhecimento previamente assimilado para tomar decisões que maximizem as recompensas baseadas no aprendizado já estabelecido pelo agente. Ao escolher o aproveitamento, um agente seleciona atividades que, conforme sua compreensão atual, possuem a capacidade de proporcionar as maiores recompensas. Essa estratégia reveste-se de relevância, especialmente quando o agente possui um conhecimento significativo sobre o ambiente e busca aproveitar os benefícios advindos de sua formação anterior.

A estratégia epsilon-greedy destaca-se como uma das abordagens mais avançadas e comumente empregadas para equilibrar essa circunstância. Sob essa ótica, o comportamento do agente é orientado por um parâmetro epsilon, que representa a probabilidade de escolher uma ação aleatória em um momento específico. Com uma probabilidade ϵ , o agente ignora completamente seu conhecimento anterior e escolhe uma ação de forma aleatória dentre todas as opções disponíveis, promovendo, assim, a exploração. Com uma probabilidade complementar igual a um menos epsilon, o agente opera de forma otimizada, escolhendo a ação que demonstra o maior valor Q estimado para o estado atual, promovendo, assim, a aplicação do conhecimento adquirido.

A elegância da estratégia epsilon-greedy reside em sua simplicidade e eficácia. Ela afirma que o agente nunca cessa completamente a exploração, preservando sempre uma probabilidade mínima de realizar ações diversas, ao mesmo tempo em que lhe propicia a oportunidade de usufruir do conhecimento acumulado na maior parte do tempo. Essa característica é essencial para evitar que o agente permaneça em ambientes favoráveis, onde uma estratégia aparentemente eficaz dificulta a identificação de táticas ainda mais benéficas.

Uma melhoria relevante na política epsilon-greedy consiste na implementação de um cronograma de redução para o valor de epsilon ao longo do tempo. Em um primeiro instante, ao encontrar um ambiente pouco conhecido, o agente sustenta epsilon em um patamar significativamente elevado, promovendo uma exploração extensa. À medida que o treinamento se desenvolve e o agente acumula experiência, o valor de epsilon é progressivamente aumentado, o que resulta na priorização do aproveitamento em relação à exploração. Esse declínio pode manifestar diferentes padrões, como linear, exponencial ou em etapas, com cada um apresentando suas vantagens específicas, de acordo com as particularidades do problema em questão.

A sincronização e a taxa de decaimento representam aspectos essenciais que requerem um metódico ajuste. Um decaimento excessivamente rápido pode levar a uma convergência prematura em uma política subótima, enquanto um decaimento excessivamente lento pode prolongar de maneira desnecessária o tempo de aprendizado e prejudicar a eficácia do agente. A escolha mais apropriada está frequentemente associada à complexidade do contexto, à densidade das recompensas e ao tempo disponível para a formação.

Além da estratégia epsilon-greedy, existem outras abordagens para equilibrar a exploração e o aproveitamento, dentre as quais se ressalta a seleção baseada na Upper Confidence Bound, a qual considera tanto o valor estimado das ações quanto a incerteza relacionada a essas estimativas. Uma alternativa é a amostragem de Thompson, que utiliza distribuições de probabilidade para expressar a incerteza relacionada aos valores das ações. Entretanto, o método epsilon-greedy permanece amplamente empregado devido à sua simplicidade de implementação e compreensão, além de sua eficácia comprovada em várias aplicações.

A gestão adequada desse equilíbrio entre exploração e aproveitamento não apenas influencia a velocidade de convergência do algoritmo Q-Learning, mas também determina a qualidade final da política obtida. Um agente que efetua uma exploração inadequada pode não reconhecer as estratégias mais eficientes disponíveis, enquanto um agente que explora em excesso pode falhar ao aproveitar o conhecimento valioso adquirido anteriormente, resultando em um desempenho inferior ao desejado, mesmo após um treinamento extensivo.

11.4. Etapas do Algoritmo

O funcionamento do algoritmo Q-Learning pode ser descrito em quatro etapas essenciais. De maneira inaugural, procede-se à inicialização, fase em que a Q-table é criada e preenchida com zeros ou valores aleatórios para cada par de estado e ação. Após a interação com o ambiente, o agente avalia o estado atual s , escolhe uma ação a , obtém uma recompensa r em função da ação realizada e observa o novo estado s' que decorre dessa transição.

Na etapa de atualização, o valor associado à entrada $Q(s, a)$ é alterado conforme a equação de Bellman, considerando tanto a recompensa imediata quanto a expectativa de recompensas futuras. Por fim, na etapa de repetição, esse ciclo é revitalizado por diversos episódios, permitindo que a Q-table avance e convirja gradualmente para uma aproximação da função ótima $Q^*(s, a)$, culminando em uma política que maximiza a recompensa total ao longo do tempo.

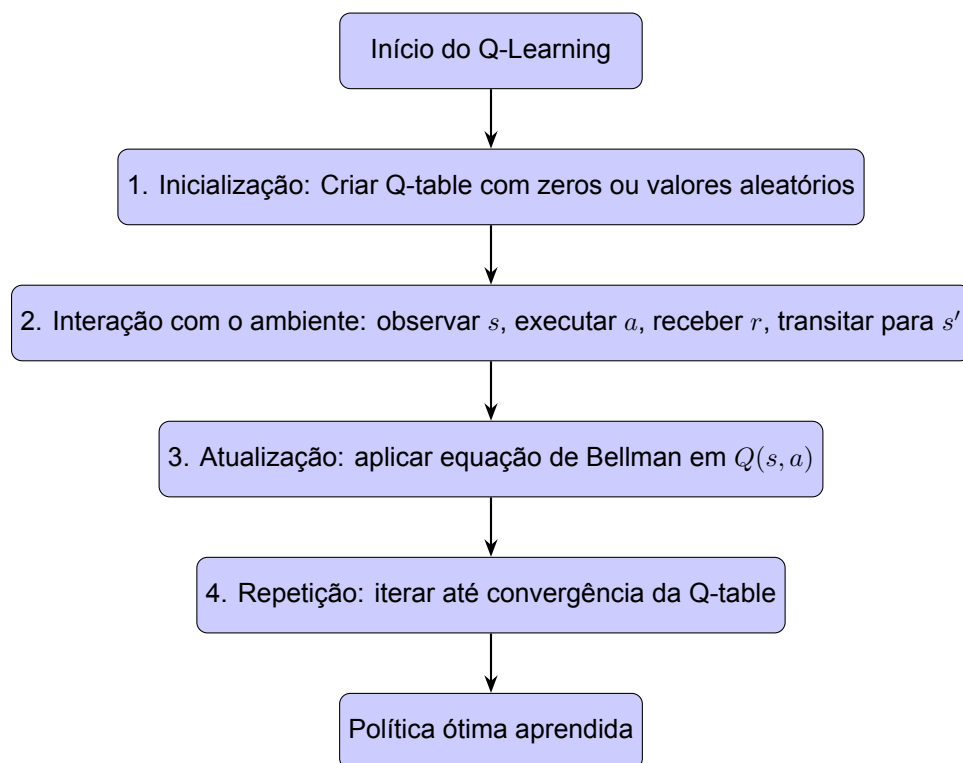


Figura 11.1: Fluxo de funcionamento do algoritmo Q-Learning.

11.5. Vantagens e Limitações do Q-Learning

O algoritmo Q-Learning apresenta diversas vantagens; não se faz necessário um modelo do ambiente. O algoritmo aprende, exclusivamente por meio da experiência acumulada nas interações com o ambiente — observando estados, escolhendo ações, bem como recebendo recompensas e transitando para novos estados. Essa particularidade o classifica como um método desprovido de um modelo específico, o que possibilita sua ampla aplicabilidade.

Além disso, sob condições ideais determinadas, como a utilização adequada do espaço de estados e a implementação de taxas de aprendizado decrescentes, o Q-Learning oferece uma garantia teórica de convergência em relação à função ótima de valor-ação Q^* . Um outro ponto positivo diz respeito à sua facilidade: a execução do algoritmo é, de modo geral, bastante simples, especialmente em contextos que dizem respeito a espaços discretos e finitos de estados e ações.

Por outro lado, o Q-Learning apresenta algumas restrições. A principal restrição que se observa refere-se à capacidade de escalabilidade. Em cenários que demonstram extensos intervalos de estados ou ações, a abordagem tabular mostra-se impraticável devido ao aumento exponencial do tamanho da tabela Q , tornando essencial, nessas condições, a adoção de técnicas mais avançadas, como o Deep Q-Learning, que utiliza redes neurais para avaliar a função Q .

Além disso, a efetividade do algoritmo é consideravelmente influenciada pela escolha adequada dos hiperparâmetros, como a taxa de aprendizado α , o fator de desconto γ e o parâmetro de exploração ϵ . A sensibilidade em relação a esses parâmetros requer cuidado durante o processo de ajuste meticuloso, visando garantir um aprendizado estável e eficaz.

Tabela 11.1: Vantagens e Desvantagens do Ambiente Determinístico

Vantagens	Limitações
Não requer modelo do ambiente: Aprende apenas com a experiência, utilizando as interações (estado, ação, recompensa, próximo estado).	Escalabilidade limitada: Em ambientes com grandes espaços de estados ou ações, a tabela Q torna-se inviável. Nestes casos, é necessário utilizar o Deep Q-Learning, que emprega redes neurais para aproximar a função Q .
Convergência garantida: Sob condições ideais de treinamento, como exploração suficiente e taxas de aprendizado adequadas, a tabela Q converge para a função ótima Q^* .	Sensibilidade a hiperparâmetros: O desempenho do algoritmo depende do ajuste apropriado dos hiperparâmetros, como a taxa de aprendizado α , o fator de desconto γ , e o parâmetro de exploração ϵ .
Simplicidade: É de fácil implementação em problemas com espaços discretos e finitos de estados e ações.	Escalabilidade: Dificuldade em ambientes com grandes espaços de estados/ações devido ao armazenamento tabular
Hiperparâmetros sensíveis: Taxa de aprendizado e fator de desconto requerem ajuste cuidadoso	

12

Aplicação Clássica do Aprendizado por Reforço

12.1. Descrição Geral do Código

Este notebook¹ implementa, passo a passo, um agente de Q-Learning no ambiente FrozenLake-v1 do Gymnasium (configurado como `is_slippery=False`, isto é, determinístico). Logo no início, são importadas as bibliotecas (Gymnasium, NumPy e Matplotlib/Animation) e fixadas sementes aleatórias para garantir a reprodutibilidade.

Em seguida, o ambiente é criado e a Q-table é inicializada com zeros no formato (`n_estados × n_ações`). Seguidamente, são definidos os hiperparâmetros do algoritmo — número de episódios, taxa de aprendizado (`alpha`), fator de desconto (`gamma`), `epsilon` e decaimento do `epsilon`, além de organizar os diretórios para salvar os artefatos (figuras, modelos e animações). Há funções utilitárias para visualização:

Mostra_Ambiente plota o mapa da grade (início, pisos, buracos e objetivo) para contextualizar o problema;

Mostra_Valores_Q converte a Q-table em um mapa de calor por estado, permitindo inspecionar a qualidade das estimativas aprendidas;

Mostra_Politica desenha a política gulosa derivada da Q-table (setas indicando a ação escolhida em cada estado); `'Gera_Video_Politica'` cria um GIF que anima a execução da política aprendida ao longo do tabuleiro.

O treinamento percorre os episódios executando uma política ϵ -greedy (com `epsilon` decaindo a cada episódio): em cada passo, o agente escolhe uma ação aleatória com probabilidade ϵ ou a melhor ação conhecida (argmax da Q-table) com probabilidade $1-\epsilon$, observa a recompensa e o novo estado e atualiza a Q-table pela equação de Bellman $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$.

¹Código Disponível no GitHub deste Artigo

A cada episódio, o notebook registra se houve sucesso (chegada ao objetivo) ou falha, e ao final exibe um gráfico de barras binário “episódios com sucesso vs. falha”, além de salvar as figuras e arquivos nos diretórios configurados.

Por fim, o notebook gera as visualizações pós-treino (mapa de valores Q, política aprendida) e, opcionalmente, a animação da política, oferecendo uma visão completa do ciclo de RL: definição do ambiente, aprendizado, inspeção das estimativas e interpretação da política resultante.

12.2. Descrevendo o Ambiente Frozenlake Utilizado

A Figura-12.1 mostra um mapa visual de um ambiente do tipo Frozen Lake, usado em problemas de aprendizado por reforço. Ele é formado por uma grade 4x4, onde cada célula possui um significado específico:

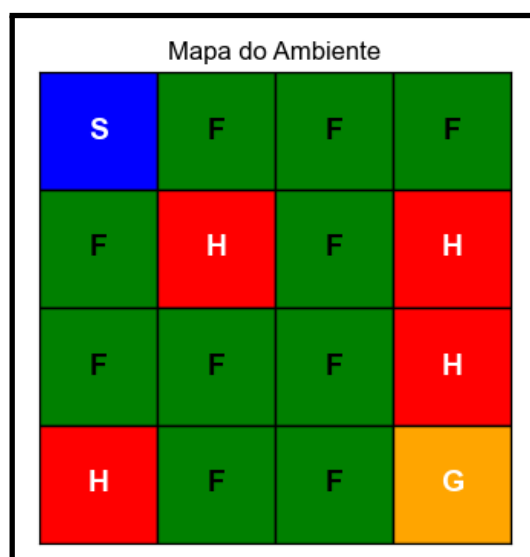


Figura 12.1: Mapa do Ambiente Frozenlake Utilizada

O mapa é composto por quatro tipos de células, cada uma com uma função específica na dinâmica do ambiente. O ponto S (Start), marcado em azul no canto superior esquerdo, indica a posição inicial do agente, de onde ele começará sua trajetória. As células F (Frozen), coloridas em verde, representam superfícies congeladas e seguras, permitindo que o agente se mova por elas sem penalidades. Já as células H (Hole), destacadas em vermelho, simbolizam buracos; se o agente cair em um deles, o episódio será encerrado de forma negativa. Por fim, a célula G (Goal), em laranja no canto inferior direito, é o objetivo final, onde o agente deve chegar para receber a recompensa máxima.

12.3. Conhecendo os Valores da Ação Ótima do Ambiente Frozenlake Utilizada

A Figura-12.2 mostra um mapa de calor com os valores ótimos da função Q para cada posição do ambiente FrozenLake, resultantes do treinamento do agente por meio do algoritmo Q-Learning.

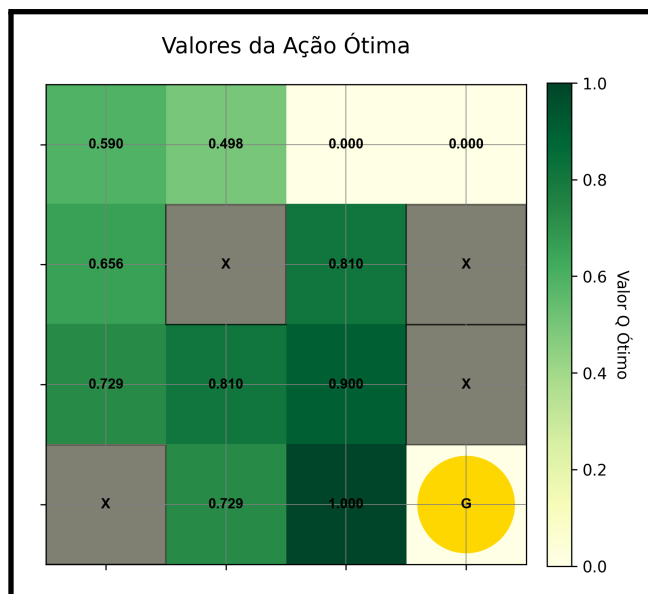


Figura 12.2: Valores Ótimos do Ambiente Frozenlake

As células verdes representam estados navegáveis (superfícies congeladas), sendo que a intensidade da cor indica o valor ótimo aprendido: quanto mais escura a tonalidade, maior o valor de Q, o que significa maior expectativa de recompensa ao seguir a política ótima a partir daquele ponto.

Os valores numéricos exibidos nas células correspondem exatamente ao valor máximo de Q obtido entre as ações possíveis naquele estado. Estados com o símbolo "X" indicam buracos (Holes), que resultam no término do episódio sem recompensa, enquanto o círculo amarelo com "G" marca a posição do objetivo final (Goal), associado ao maior valor possível (1.000).

O padrão observado revela a trajetória mais vantajosa que o agente aprendeu: valores crescentes de Q à medida que se aproxima do objetivo, refletindo o cálculo recursivo da equação de Bellman e a política determinística sem exploração utilizada no treinamento.

12.4. Política Aprendida no Ambiente Frozenlake Utilizada

A Figura-12.3 mostra a política aprendida para o ambiente FrozenLake após o treinamento com Q-Learning. Cada célula branca contém uma seta azul indicando a ação que o agente deve tomar a partir daquele estado para seguir a rota considerada ótima segundo a política final.

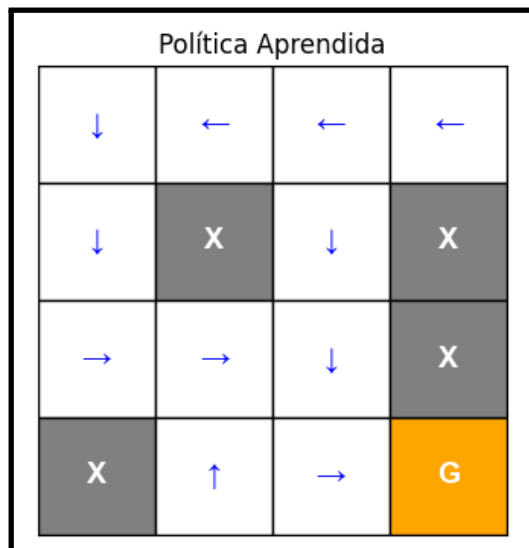


Figura 12.3: Política Aprendida no Ambiente Frozenlake

As células cinzas marcadas com “X” representam os buracos (Holes), que são estados terminais indesejados onde o episódio é perdido caso o agente entre neles. No canto inferior direito, a célula laranja marcada com “G” representa o objetivo (Goal), que concede a recompensa máxima quando alcançado.

Observa-se que a política aprendida conduz o agente desde o estado inicial (fora da imagem, mas normalmente no canto superior esquerdo do mapa) até o objetivo, evitando os buracos e traçando um caminho seguro, geralmente passando pela segunda e terceira fileiras. O fluxo de setas indica que o agente aprendeu a seguir sempre pelas rotas de maior valor estimado, contornando os riscos e garantindo a maior probabilidade de sucesso.

12.5. Episódios com Sucesso e Falha do Ambiente Frozenlake Utilizada

A Figura-12.4 ilustra a evolução do desempenho do agente ao longo de 1000 episódios no ambiente Frozen Lake, distinguindo visualmente os episódios de sucesso (representados pelo valor 1 no eixo y) daqueles de falha (valor 0).

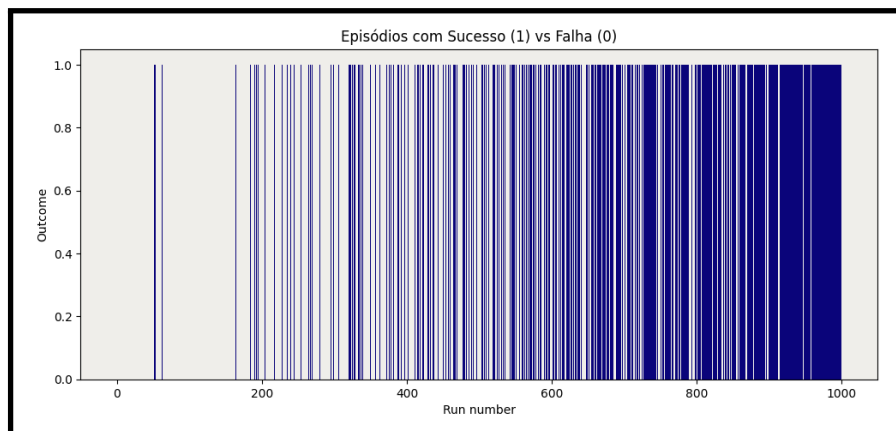


Figura 12.4: Episódios de Sucesso e Falha no Ambiente Frozenlake

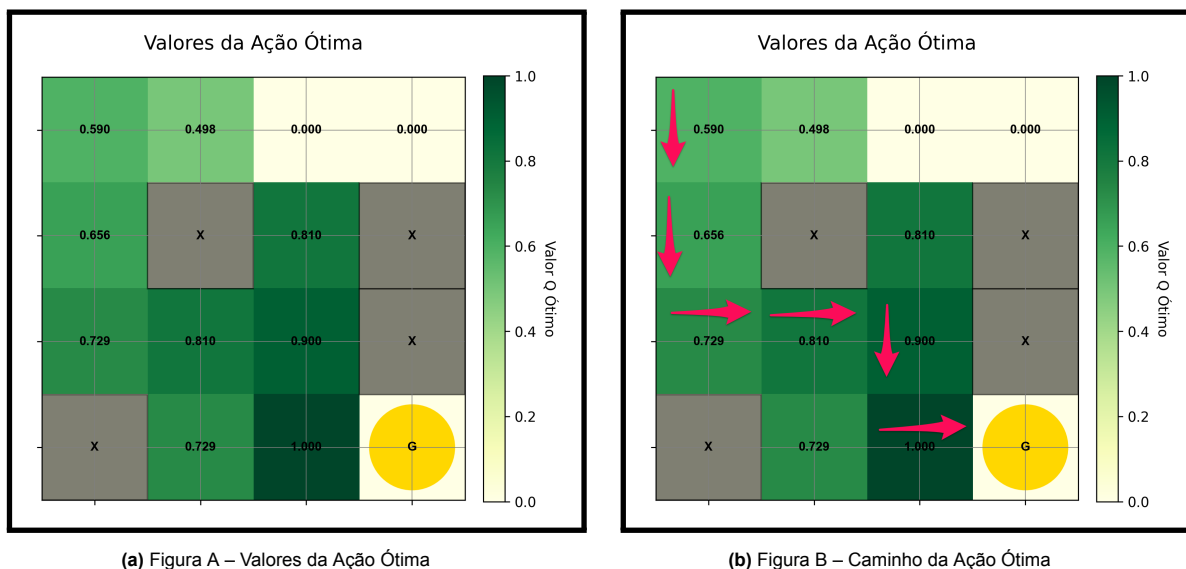
No início do treinamento, observam-se longos intervalos com ausência de sucessos, o que reflete a fase inicial de aprendizado, na qual o agente ainda não domina a política para alcançar o objetivo. Conforme o treinamento avança, há um aumento gradual na frequência de episódios bem-sucedidos, indicado pela maior densidade de linhas correspondentes ao valor 1.

Esse padrão evidencia o processo de ajuste da Q-table e a melhora da política ao longo do tempo, até que, nas iterações finais, o agente atinge um desempenho mais consistente, com sucessos ocorrendo de forma muito mais frequente e regular.

12.6. Critérios Usados para a Construção da Política

A política obtida na segunda imagem foi formada a partir dos valores ótimos de Q mostrados na primeira figura, seguindo o princípio central do Q-Learning: para cada estado, escolhe-se a ação que maximiza o valor esperado de recompensa futura.

No mapa, as setas indicam o caminho que o agente deve seguir para alcançar o objetivo (G) com a maior probabilidade de sucesso, evitando estados perigosos (X), que representam buracos.



O critério de formação dessa política considera que, para cada célula acessível, o algoritmo avalia todas as ações possíveis e seleciona aquela cujo valor Q é o maior, indicando ser a mais promissora para chegar ao objetivo.

Como resultado, observamos um trajeto contínuo que começa na posição inicial, passa por estados com altos valores de Q e contorna os obstáculos, terminando no estado final com valor Q igual a 1, representando o máximo de recompensa.

13

Aplicando o Aprendizado por Reforço em Sistemas de Recomendação

13.1. Descrição Geral do Código

O código “APR - Q Learning - Sistema de Recomendação - Versão 1.2.0”¹ tem como objetivo implementar um sistema de recomendação utilizando o algoritmo Q-Learning no contexto de aprendizado por reforço. A proposta central é treinar um agente para identificar ações que maximizem recompensas cumulativas, adaptando-se ao comportamento do usuário e às características do ambiente.

Esse tipo de abordagem é especialmente útil em cenários nos quais as recomendações dependem da interação contínua entre o sistema e o usuário, permitindo que o modelo aprenda com a experiência acumulada ao longo do tempo. De forma geral, o código é estruturado para:

Definir e inicializar o ambiente estabelecendo os estados e ações possíveis, além das regras que governam as transições entre estados e as recompensas associadas.

Configurar os parâmetros do Q-Learning define como taxa de aprendizado (α), fator de desconto (γ) e política de exploração (mesmo que, nesta versão, possa ser ajustada para diferentes graus de exploração/exploração).

Treinar o agente atualiza a matriz Q a cada interação com o ambiente, de acordo com a equação de Bellman, para convergir para uma política ótima.

Gerar e avaliar recomendações aplicação a política aprendida para sugerir itens ou ações que maximizem a recompensa esperada, com base no histórico de interações.

Assim, o programa atua como uma simulação prática de como o Q-Learning pode ser aplicado em sistemas de recomendação, destacando a importância da aprendizagem adaptativa e do ajuste dinâmico das estratégias de decisão conforme as respostas do usuário e as mudanças no ambiente.

¹Código Disponível no GitHub deste Artigo

13.2. Distribuição dos Cliques por Segmentos

A Figura-13.1 mostra um gráfico de radar ilustrando a distribuição dos cliques por segmento, categorizando-os em cinco áreas de interesse: world, sports, tech, finance e entertainment. Cada linha colorida representa um segmento distinto de usuários: Segmento 0 (azul), Segmento 1 (laranja), Segmento 2 (verde) e Segmento 3 (vermelho), permitindo uma comparação visual de seus comportamentos.

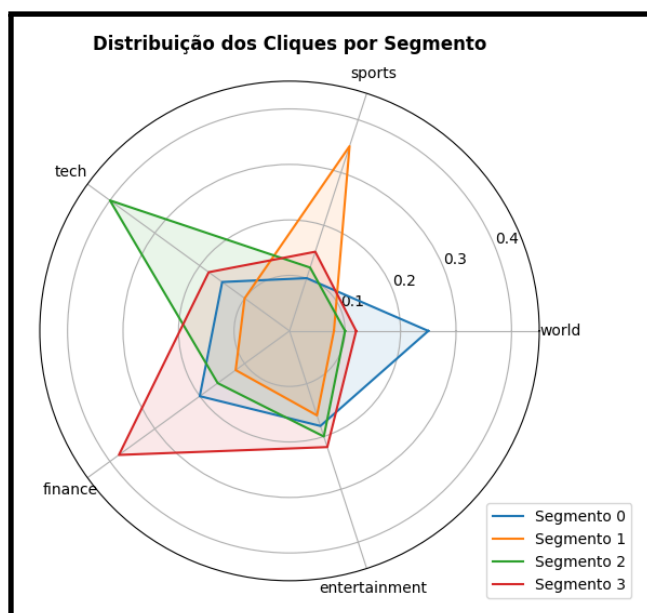


Figura 13.1: Distribuição dos Cliques por Segmentos

Observa-se que o Segmento 0 apresenta maior concentração de cliques na categoria world, com presença moderada em finance e entertainment, mas baixa em sports e tech. O Segmento 1 se destaca em sports, sendo o grupo com maior afinidade nessa categoria, enquanto mantém valores reduzidos em world, tech e finance.

Já o Segmento 2 possui pico expressivo em tech, indicando um forte interesse por tecnologia, e valores intermediários nas demais áreas. Por fim, o Segmento 3 concentra a maior parte dos cliques em finance, seguido de participação equilibrada em entertainment e tech, mas com menor envolvimento em world e sports.

Esse padrão visual evidencia que cada segmento possui preferências temáticas distintas, o que pode ser explorado para estratégias de personalização de conteúdo e recomendações direcionadas, aumentando a relevância e o engajamento dos usuários em cada grupo.

13.3. Comportamento do Click-Through Rate Usado no Treinamento

A Figura-13.2 ilustra a variação do *CTR* (Click-Through Rate) ao longo de 2000 episódios de treino. No eixo horizontal, encontram-se os episódios, enquanto o eixo vertical representa os valores de *CTR*, que variam aproximadamente entre 0,05 e 0,58. A série demonstra oscilações constantes, sem uma tendência clara de crescimento ou queda acentuada, sugerindo que, ao longo do treinamento, o desempenho em termos de taxa de cliques manteve-se dentro de um intervalo relativamente estável, ainda que com flutuações pontuais.

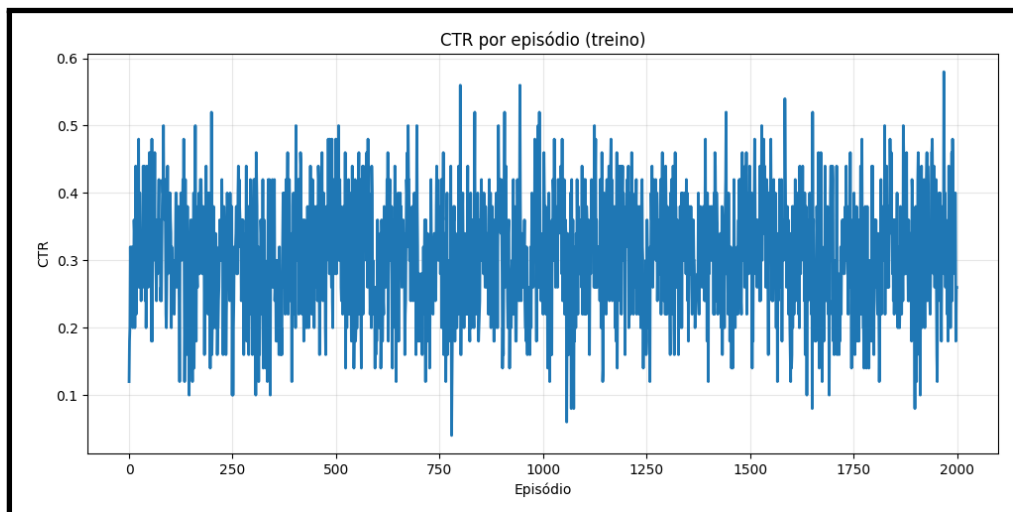


Figura 13.2: CTR por Episódio Utilizado no Treinamento

O **CTR (Click-Through Rate)** é uma métrica utilizada para medir a taxa de cliques em relação ao número de exibições (impressões) de um item, como um anúncio, recomendação ou link. É calculado pela fórmula:

$$CTR = \frac{\text{número de cliques}}{\text{número de impressões}}$$

Em sistemas de recomendação e aprendizado por reforço, o *CTR* é um indicador importante da eficácia das ações do agente, pois reflete o engajamento real do usuário com as sugestões apresentadas. Valores mais altos de *CTR* indicam que as recomendações ou ações propostas são mais relevantes ou atraentes para os usuários, enquanto valores baixos apontam para a necessidade de ajustes na política de decisão.

13.4. Decaimento da Exploração ao Longo do Treinamento

A Figura-13.3 ilustra o comportamento do parâmetro de exploração ϵ ao longo dos episódios de treinamento. Inicialmente, observa-se um valor elevado de aproximadamente 0,87, o que indica que, nas primeiras interações, o agente adota uma política predominantemente exploratória, escolhendo ações de forma mais aleatória para coletar informações sobre o ambiente. Logo nos primeiros episódios, há uma queda acentuada de ϵ até atingir um patamar próximo de 0,05, que permanece praticamente constante durante o restante do treinamento.

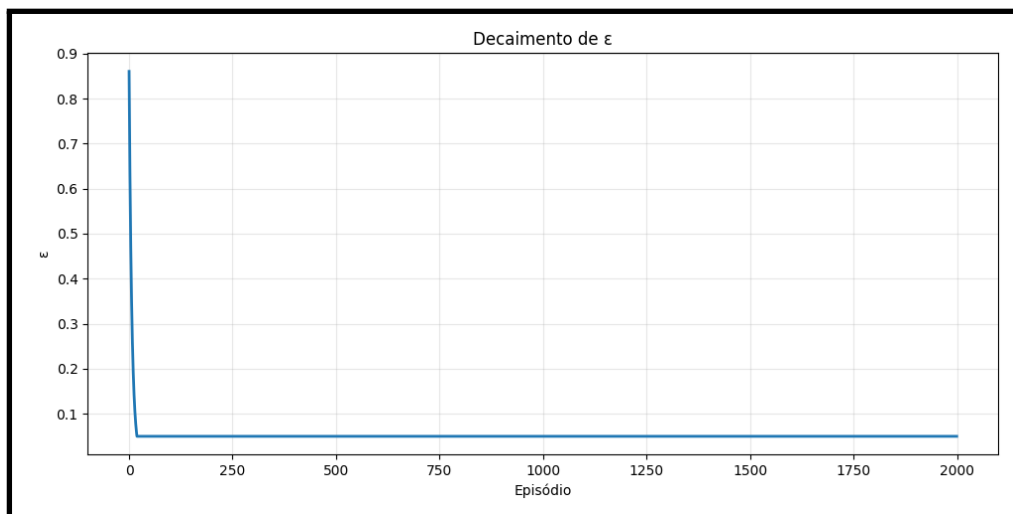


Figura 13.3: Decaimento do Epsilon ao Longo do Treinamento

Esse comportamento reflete uma estratégia de decaimento rápido do ϵ , na qual a exploração é priorizada apenas no início do aprendizado, permitindo que o agente, após adquirir conhecimento suficiente, passe a explorar menos e aproveitar mais as ações com maiores valores estimados, favorecendo a convergência para uma política ótima.

13.5. Aprendizado da Tabela Q

A Figura-13.4 mostra o heatmap apresentado ilustra a Tabela Q aprendida ao longo do processo de treinamento, representando a relação entre estados (no eixo vertical) e ações possíveis (no eixo horizontal). Cada célula contém o valor Q estimado para a combinação específica de estado e ação, onde valores mais altos indicam uma expectativa maior de retorno futuro ao executar aquela ação no estado correspondente.

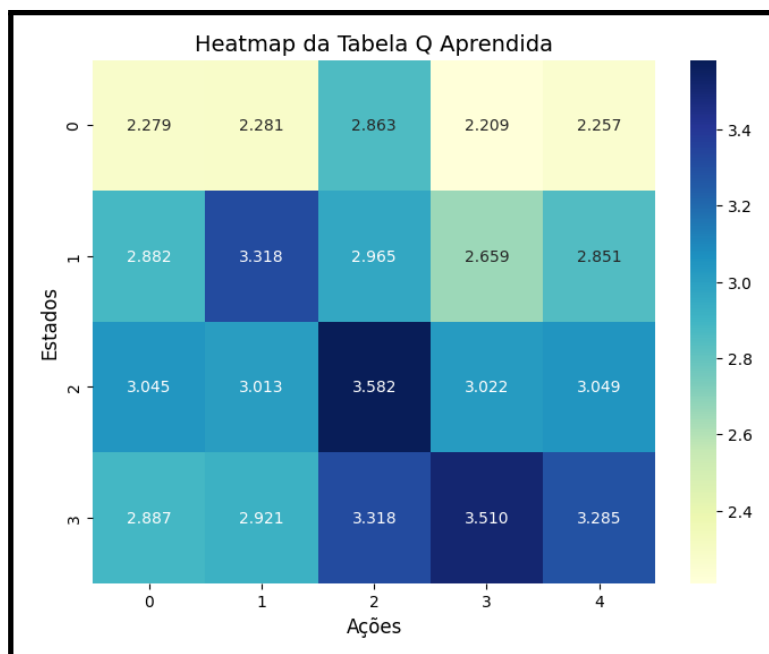


Figura 13.4: Heatmap da Tabela Q Aprendida

A escala de cores, que vai de tons claros (valores mais baixos) a tons escuros (valores mais altos), facilita a identificação visual das combinações mais promissoras. Observa-se que, no estado 2, a ação 2 apresenta o maior valor Q (3,582), destacando-se como a mais vantajosa nesse contexto, enquanto no estado 3, a ação 3 também se sobressai com um valor de 3,510.

Essa representação é útil para interpretar a política aprendida, pois evidencia as decisões que o agente tende a priorizar para maximizar a recompensa cumulativa.

13.6. Resultados Obtidos

A Tabela-13.1 apresenta informações sobre diferentes usuários, seus segmentos e as pontuações atribuídas a cada categoria de conteúdo.

Tabela 13.1: Resultados por Usuário e Segmento

Usuário	Segmento	World	Sports	Tech	Finance	Entertainment
Ana	2	3.0445		3.5823		3.0494
Bruno	1	2.8815	3.3182	2.9647		
Carla	M			3.1989	2.9372	2.8864
Diego	3			3.3180	3.5103	3.2854
Eva	M		2.6685	3.1342		2.6572

Ana pertence ao segmento 2 e possui pontuação 3,0445 em World, 3,5823 em Tech e 3,0494 em Entertainment. Bruno, do segmento 1, tem 2,8815 em World, 3,3182 em Sports e 2,9647 em Tech. Carla, identificada pelo segmento “M”, registra 3,1989 em Tech, 2,9372 em Finance e 2,8864 em Entertainment. Diego, do segmento 3, apresenta 3,3180 em Sports, 3,5103 em Finance e 3,2854 em Entertainment. Por fim, Eva, também do segmento “M”, possui 2,6685 em Sports, 3,1342 em Tech e 2,6572 em Entertainment.

Os dados mostram a distribuição de preferências ou relevâncias por categoria, permitindo identificar padrões de interesse por segmento e usuário individualmente.

13.7. Porque Usar um IF/THEN/ELSE não é a Melhor Solução

Ao analisar a tabela apresentada e a lista de usuários com seus respectivos tipos de recomendação — “segment” ou “mixture” — percebe-se que utilizar apenas um simples comando condicional ‘if’ para decidir a lógica não é a melhor abordagem. Essa escolha traria limitações significativas em diferentes aspectos.

Primeiramente, do ponto de vista de escalabilidade e manutenção, o uso de estruturas condicionais ‘if/elif’ para cada tipo de recomendação ou segmento faria o código crescer rapidamente à medida que novos tipos ou combinações fossem adicionados. Esse modelo gera um “código espaguete” difícil de manter e de expandir, pois qualquer inclusão futura exigiria alterações diretas na lógica central, criando novas condicionais e aumentando o risco de erros.

Além disso, existe a questão da flexibilidade de dados. Usuários do tipo “mixture” possuem pesos diferentes para cada segmento — como no caso de Carla e Eva — e, portanto, uma lógica rígida baseada apenas em ‘if segment_id == X’ não funcionaria. Esses casos não se enquadram em um único valor fixo, e sim em distribuições ponderadas que precisam ser processadas de forma dinâmica, o que demanda uma abordagem mais adaptável.

Outro ponto relevante é a abordagem baseada em dados. A tabela apresenta valores para diferentes categorias — World, Sports, Tech, Finance e Entertainment — que variam para cada usuário. Em vez de condicionar todas as decisões em blocos ‘if’, é mais eficiente criar funções capazes de ler e interpretar esses valores diretamente, adaptando as recomendações de acordo com o perfil do usuário, seja ele do tipo “segment” ou “mixture”. Essa estratégia elimina a necessidade de regras fixas no código e possibilita trabalhar de forma mais integrada com as estruturas de dados.

Por fim, sob a ótica das boas práticas de engenharia de software, é fundamental manter a separação de responsabilidades, isolando a lógica de seleção do tipo de recomendação do cálculo dos valores. Ao invés de utilizar condicionais múltiplas, pode-se empregar técnicas de generalização, como mapeamento — por exemplo, dicionários que associem ‘segment_id’ a segmentos — ou polimorfismo, com funções distintas para cada tipo de recomendação. Essa abordagem também favorece a facilidade de teste, uma vez que menos condicionais fixas resultam em funções mais previsíveis e de comportamento controlado.

14

Aplicando o Aprendizado por Reforço na Gestão de Energia

14.1. Descrição Geral do Código

O notebook APR – Q Learning – Gestão de Energia – Versão 1.0.0¹ tem como propósito aplicar o algoritmo de Q-Learning em um cenário voltado para a otimização do consumo energético. Ele cria um ambiente de simulação no qual diferentes cargas ou dispositivos podem ser ligados ou desligados de forma inteligente, com o objetivo de minimizar custos sem violar restrições de operação impostas ao sistema.

Nesse contexto, o problema é modelado como um Processo de Decisão de Markov (MDP). Os estados representam o tempo e a duração restante de cada carga, enquanto as ações correspondem às decisões de ligar ou desligar dispositivos em cada instante.

As recompensas refletem o impacto econômico dessas decisões, podendo traduzir tanto custos quanto economias obtidas. Essa formulação permite que o agente aprenda, ao longo do tempo, políticas de acionamento mais eficientes, capazes de maximizar a utilização racional de energia.

Durante a etapa de simulação e resultados, o notebook executa múltiplos episódios de treinamento, atualizando continuamente a Q-table. Ao final, essa tabela apresenta valores otimizados que indicam a política aprendida, ou seja, a melhor sequência de ações de liga/desliga dos dispositivos em função do estado em que o sistema se encontra.

Os resultados mostram como o agente é capaz de equilibrar custo e tempo de operação, identificar quais ações são mais vantajosas em determinados estados e evidenciam a convergência do aprendizado após diversas interações.

¹Código Disponível no GitHub deste Artigo

Além disso, o código disponibiliza recursos de visualização que tornam mais clara a análise do desempenho. São apresentados gráficos com a evolução das recompensas ao longo dos episódios, mapas de decisão que ilustram as políticas aprendidas e comparações entre diferentes estratégias de acionamento de cargas. Essas representações permitem avaliar de forma intuitiva a efetividade do Q-Learning na gestão de energia.

Assim, o notebook oferece uma aplicação prática do Q-Learning para problemas de eficiência energética, simulando o controle de dispositivos sujeitos a restrições de duração e custo. Ao unir conceitos de aprendizado por reforço e otimização operacional, o código funciona como um ponto de partida sólido para explorar cenários mais complexos de automação energética em ambientes reais.

14.2. Aprendizado da Tabela Q

O gráfico apresentado mostra a Figura-14.1 após o treinamento do algoritmo de Q-Learning aplicado ao problema de gestão de energia. Cada subgráfico corresponde a uma ação específica: Ação 0 (nenhuma carga ligada), Ação 1 (ligar a carga A), Ação 2 (ligar a carga B) e Ação 3 (ligar a carga C)

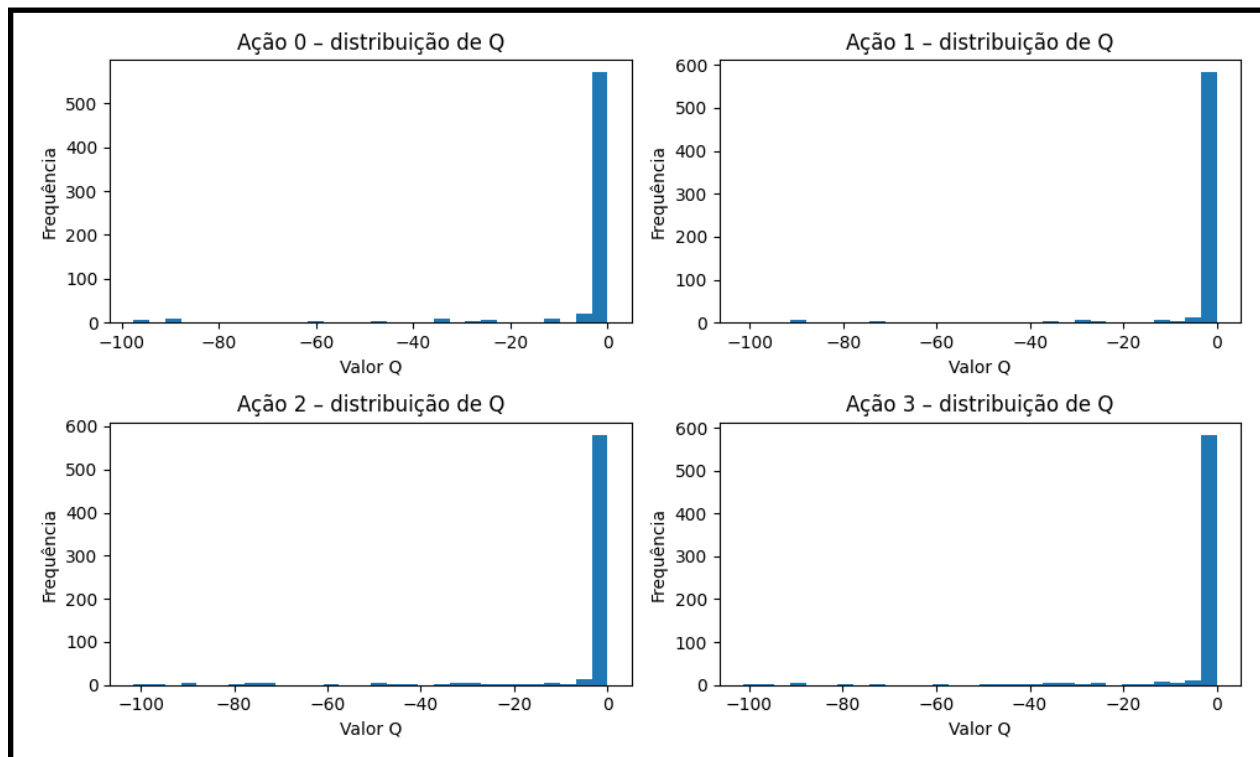


Figura 14.1: Comportamento da Tabela Q Aprendida

A maior parte dos valores de Q está concentrada próxima de zero em todas as ações. Isso indica que, em muitos estados, as decisões do agente não produzem ganhos significativos ou apresentam recompensas neutras. É esperado em problemas de otimização energética, já que nem sempre ligar ou desligar um dispositivo gera impacto relevante no custo.

Sobre os Valores Negativos, observa-se que algumas ações possuem valores Q negativos expressivos, chegando a valores próximos de -100. Esses casos representam situações em que a decisão tomada leva a um custo elevado ou a uma estratégia pouco eficiente, como ligar cargas em horários com preço alto de energia. Isso mostra que o agente aprendeu a penalizar essas escolhas ao longo do treinamento.

O acúmulo de valores próximos de zero sugere que o agente muitas vezes se encontra em estados onde a melhor ação é não realizar mudanças drásticas (ação 0, manter o sistema como está), evitando penalidades e mantendo o equilíbrio no consumo.

Embora os quatro gráficos apresentem um padrão semelhante, percebe-se que algumas ações (1, 2 e 3) apresentam mais dispersão nos valores negativos. Isso reforça que ligar cargas em momentos inadequados pode resultar em custos adicionais, enquanto manter o sistema estável tende a ser uma estratégia mais segura.

O resultado da Q-table mostra que o agente aprendeu a distinguir claramente situações vantajosas de desvantajosas. A predominância de valores próximos de zero indica cautela e estabilidade no controle energético, enquanto os valores negativos refletem a penalização de decisões ruins. Essa distribuição confirma que o Q-Learning foi capaz de capturar o balanço entre economia de energia e custo de operação no ambiente simulado.

14.3. Interpretando os Resultados do Treinamento

A Figura-14.2 mostra a evolução do custo por episódio durante o treinamento do agente de Q-Learning aplicado à gestão de energia. Cada barra azul representa o custo obtido em um episódio individual, enquanto a linha laranja mostra a média móvel (50 episódios), funcionando como um indicador de tendência ao longo do tempo.

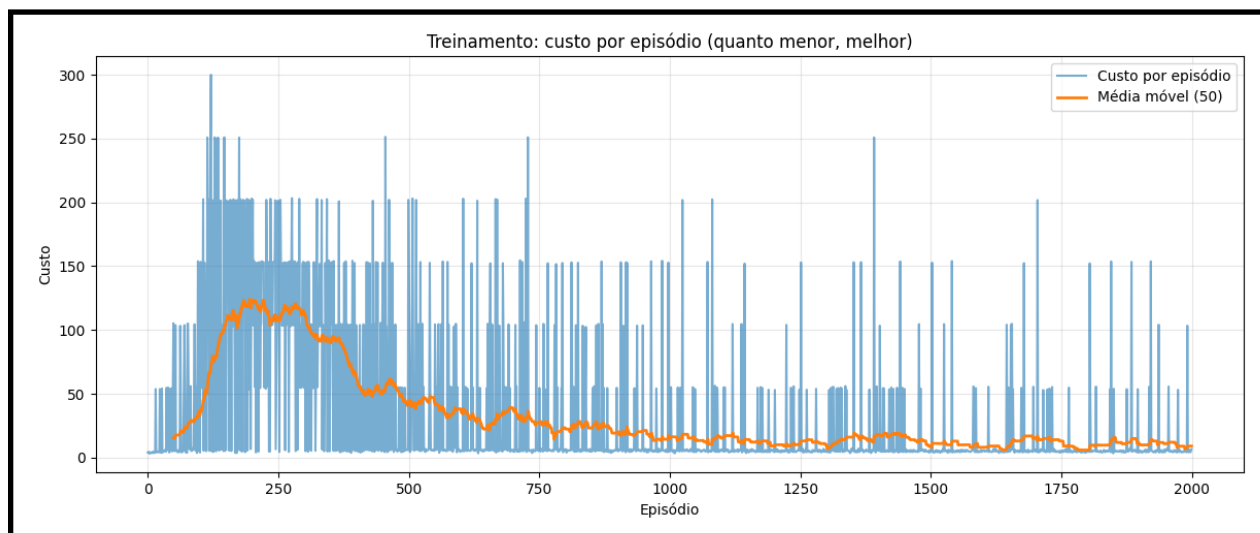


Figura 14.2: Interpretando os Resultados do Treinamento

No início do treinamento, observa-se uma grande variação no custo, com valores que chegam a ultrapassar 250 unidades. Isso acontece porque o agente ainda está explorando o ambiente e testando diferentes sequências de ações sem ter aprendido uma estratégia eficiente. Ao longo dos episódios, nota-se que a linha de tendência começa a cair gradualmente, mostrando que o agente está aprendendo a reduzir os custos ao adotar políticas mais adequadas.

A partir de aproximadamente 500 episódios, o custo médio passa a se estabilizar em valores bem mais baixos, com picos ocasionais, mas mantendo uma tendência decrescente até próximo de zero em várias execuções. Isso indica que o agente conseguiu aprender uma política eficiente para acionar os dispositivos de forma otimizada, respeitando restrições e reduzindo o custo de operação energética.

Os picos intermediários observados mesmo após a estabilização estão relacionados ao equilíbrio entre exploração e exploração. Durante o treinamento, o agente ainda precisa explorar novas sequências de ações para confirmar que as escolhas atuais são realmente ótimas. Essa exploração pode levá-lo temporariamente a decisões subótimas, aumentando o custo em alguns episódios. No entanto, com a continuidade do aprendizado, essas variações diminuem de impacto, e o agente reforça a política mais eficiente encontrada.

O gráfico mostra o processo de aprendizado do agente começando com alta variabilidade e custos elevados, mas, com a experiência acumulada, converge para uma estratégia mais estável e de baixo custo, demonstrando a efetividade do Q-Learning no problema de gestão de energia, ao mesmo tempo em que os picos intermediários ilustram o papel da exploração no refinamento da política aprendida.

14.4. Resultado do Treinamento

A Figura-14.3 mostra o resultado do aprendizado por reforço aplicado ao problema de gestão de energia, representado em forma de gráfico de dispersão categórica. No eixo horizontal temos as horas do dia (de 0 a 23), enquanto no eixo vertical estão as ações possíveis que o agente pode tomar: None (nenhuma ação), A, B ou C.

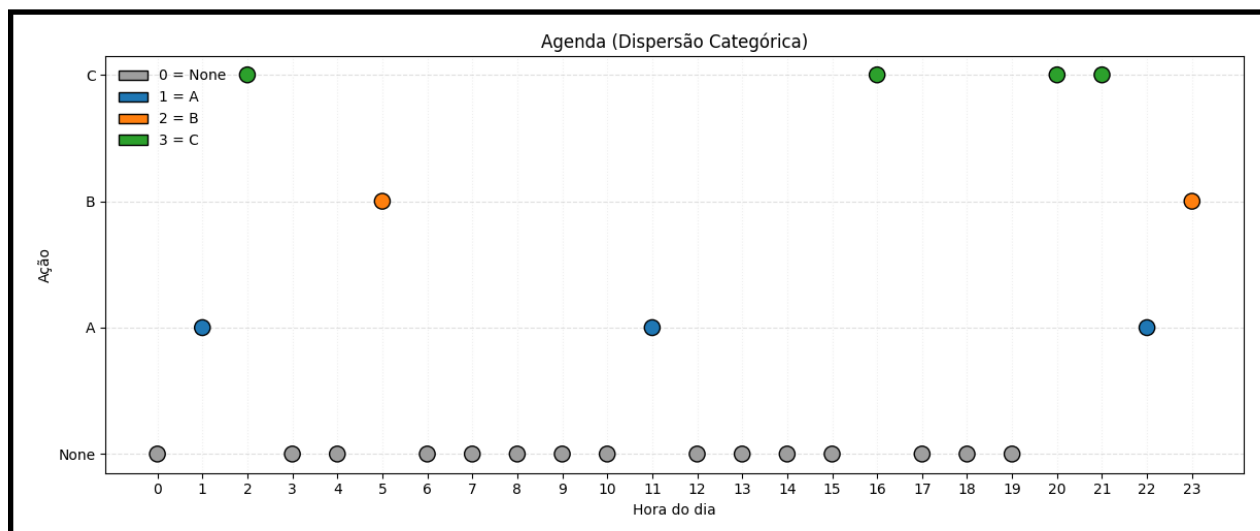


Figura 14.3: Resultados do Treinamento

Cada ponto no gráfico indica a decisão tomada pelo agente em determinada hora. As cores ajudam a diferenciar visualmente as ações: cinza para None, azul para A, laranja para B e verde para C.

O resultado mostra que, na maior parte do tempo, a ação escolhida é None, ou seja, o agente opta por não ativar nenhuma carga. Isso é esperado, pois o aprendizado prioriza a minimização de custos e só liga dispositivos em momentos considerados vantajosos.

Em horários específicos, como 1h, 11h, 22h e outros, aparecem as ativações das cargas A, B e C, distribuídas de acordo com o padrão aprendido pelo algoritmo. A presença dessas ativações pontuais indica que o agente aprendeu a conciliar o funcionamento das cargas com os preços por hora e a duração de cada dispositivo, de modo a garantir a eficiência energética.

Esse tipo de visualização é importante porque mostra, de forma direta, a política aprendida pelo agente, ou seja, quando ligar cada dispositivo e quando permanecer inativo. Assim, podemos verificar se o algoritmo encontrou um padrão coerente e interpretar sua lógica de decisão em termos práticos para gestão de energia.

14.5. Conclusão

A análise integrada dos resultados demonstra que o Q-Learning foi capaz de aprender uma política eficiente de gestão de energia, equilibrando custo e funcionamento das cargas.

Os histogramas de Q revelam a consolidação dos valores em torno de ações de baixo custo; o gráfico de custo por episódio mostra a convergência clara do aprendizado; e a dispersão categórica evidencia uma agenda de acionamento otimizada, com ativações pontuais nos horários mais adequados.

Assim, o modelo valida o potencial do aprendizado por reforço como ferramenta prática para problemas de eficiência energética, servindo de base para cenários mais complexos de automação e controle.

15

Aplicando o Aprendizado por Reforço para Aplicação Financeira

15.1. Descrição Geral do Código

O notebook APR – Q Learning – Sistema Financeiro – Versão 1.1.0 implementa um experimento de *Aprendizado por Reforço* utilizando o algoritmo Q-Learning aplicado ao contexto de operações financeiras. O objetivo central é comparar a política aprendida pelo agente com a estratégia tradicional de buy & hold.

O fluxo inicia com a obtenção das séries históricas (preço de fechamento/ajustado e, quando disponível, volume) e a engenharia de atributos, responsáveis por compor o estado do agente. Entre esses atributos estão retornos normalizados, médias móveis e indicadores técnicos como RSI. Após a preparação, a série é dividida em dois conjuntos: treino e teste, para aprendizado e avaliação fora da amostra, respectivamente.

O ambiente é modelado em passos temporais discretos, onde: os estados descrevem o contexto de mercado em um instante t , incluindo posição atual (comprado, vendido ou neutro). As ações correspondem a {Comprar, Vender, Manter} ou variações simplificadas como abrir/fechar posição. As recompensas refletem o retorno incremental gerado pela ação, considerando custos de transação e slippage.

A avaliação do modelo ocorre após o treinamento, o agente é avaliado no conjunto de teste, aplicando apenas a política aprendida. São calculadas métricas como: Retorno acumulado, Máximo Drawdown (MDD) e Índice de Sharpe.

A visualização e resultados, o notebook produz gráficos como: Curva de patrimônio (equity) do agente comparada ao buy & hold, Evolução de ϵ e recompensas por episódio, além da Distribuição de retornos ou heatmaps da Q-table.

15.2. Decaimento do Parâmetro de Exploração ϵ

A Figura-15.1 mostra o decaimento do parâmetro de exploração ϵ ao longo do processo de treinamento do agente de aprendizado por reforço. No início, ϵ possui um valor relativamente alto, permitindo maior exploração de ações aleatórias para que o agente conheça melhor o ambiente. Logo nas primeiras iterações, ocorre uma queda acentuada de ϵ , que rapidamente se estabiliza em torno de 0,05.

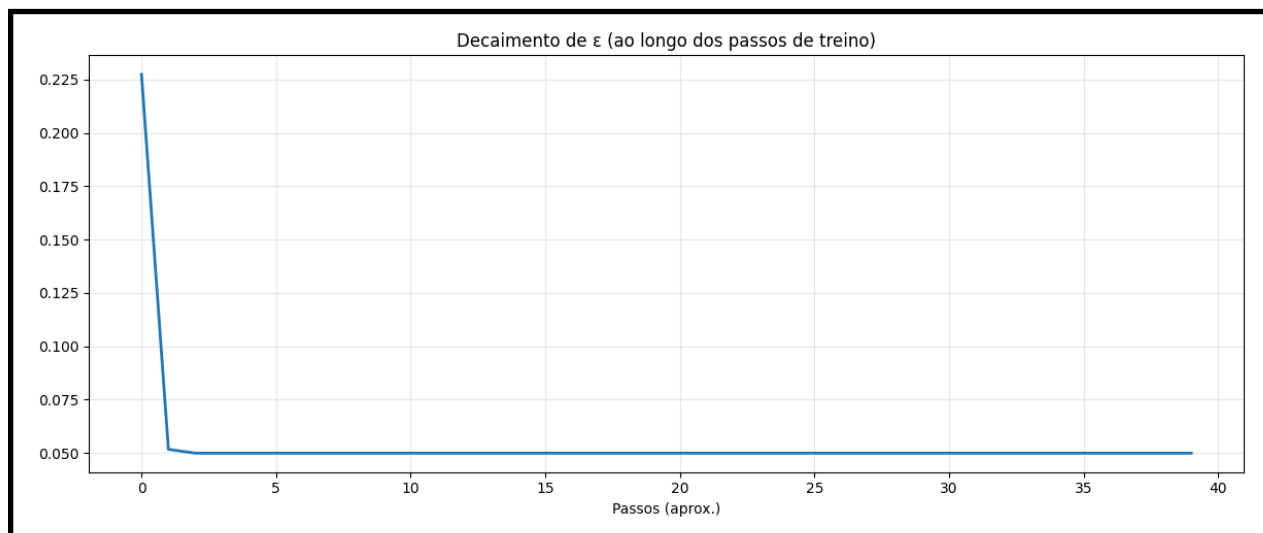


Figura 15.1: Decaimento do Parâmetro de Exploração

Isso indica que, após essa fase inicial, o agente praticamente abandona a exploração e passa a agir de forma quase determinística, escolhendo majoritariamente as ações com maior valor aprendido na Q-table. Esse comportamento demonstra uma estratégia em que a exploração intensa é restrita ao início do treinamento, favorecendo a rápida convergência da política aprendida, ainda que com risco de limitar a descoberta de alternativas potencialmente melhores em fases posteriores.

15.3. Aprendizado da Tabela Q

A Figura-15.2 mostra um heatmap da Q-Table, que representa os valores aprendidos pelo agente de Q-Learning em função dos estados e das ações disponíveis. No eixo vertical estão os estados, definidos pela combinação de duas variáveis discretizadas (posição e momentum, neste caso binarizados), enquanto no eixo horizontal estão as três ações possíveis: 0 = Flat (não tomar posição), 1 = Long (comprar) e 2 = Short (vender). A escala de cores, indicada pela barra lateral, mostra a intensidade dos valores Q, com tons mais próximos do amarelo indicando valores mais altos, ou seja, maior expectativa de recompensa, e tons mais escuros indicando valores menores.

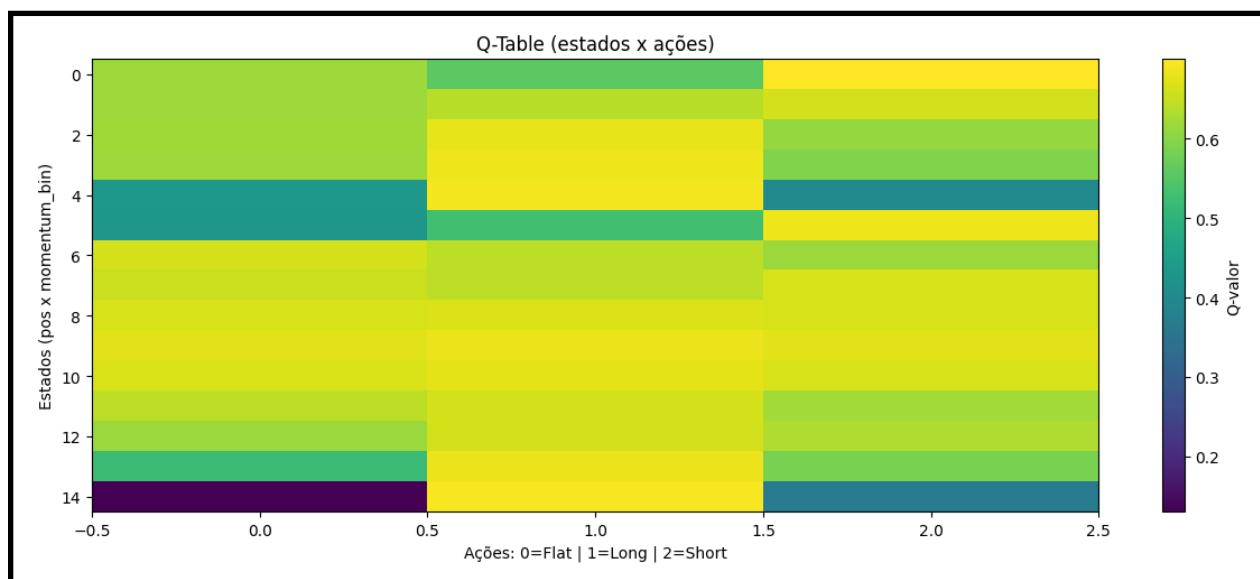


Figura 15.2: Aprendizado da Tabela Q

Observando a distribuição, percebe-se que, para diversos estados, os valores de Q associados às ações Long e Short são mais elevados do que para a ação Flat, sugerindo que a política aprendida tende a favorecer operações ativas em vez de se manter neutra. Há, entretanto, estados onde a ação Flat possui valores comparáveis ou até superiores, indicando que, em certas condições, permanecer sem posição é a estratégia ótima para minimizar perdas ou evitar decisões arriscadas.

Outro ponto relevante é a presença de variação significativa entre estados, mostrando que a política não é uniforme: o agente aprendeu a distinguir situações mais favoráveis para operar comprado, vendido ou manter-se neutro, de acordo com as características do ambiente. Isso reflete o processo de convergência do Q-Learning, que busca atribuir valores às combinações estado-ação de acordo com as recompensas observadas ao longo do treinamento.

Em resumo, o heatmap mostra que a Q-Table foi preenchida com padrões diferenciados, evidenciando que o agente aprendeu uma estratégia adaptativa. A cor predominante em certos estados indica quais ações são consideradas mais promissoras, permitindo extrair a política final que maximiza a recompensa acumulada. Esse resultado reforça que o algoritmo conseguiu capturar as dinâmicas do ambiente, diferenciando contextos em que é melhor comprar, vender ou permanecer estável.

15.4. Interpretando os Resultados do Treinamento

A Figura-15.3 mostra uma comparação entre duas estratégias aplicadas à série histórica da ação PETR4.SA: uma baseada em Q-Learning e outra no método tradicional de Buy & Hold.

O eixo horizontal representa o tempo em dias úteis de negociação, enquanto o eixo vertical mostra o capital normalizado, ou seja, a evolução do patrimônio inicial ao longo do período de teste.

A curva azul corresponde ao desempenho do agente treinado com Q-Learning, que toma decisões dinâmicas de compra, venda ou permanência em função da política aprendida.

A curva laranja representa a estratégia de Buy & Hold, onde o investidor compra a ação no início do período e a mantém até o final, sem ajustes.

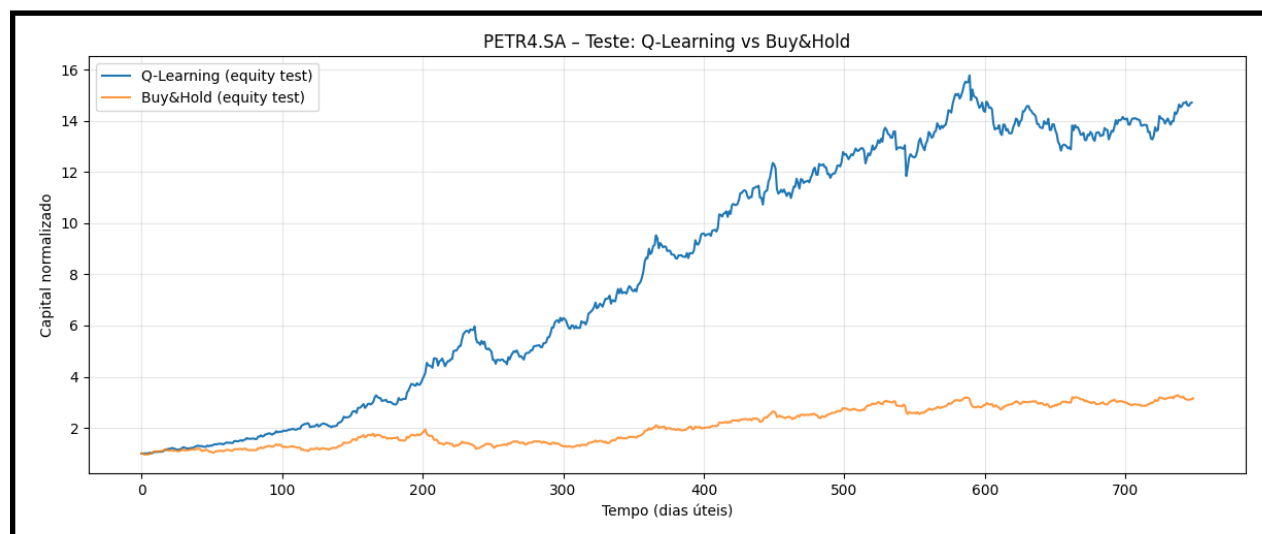


Figura 15.3: Comportamento da Tabela Q Aprendida

Observa-se que, enquanto o Buy & Hold apresentou um crescimento modesto e consistente, atingindo cerca de 3,2 vezes o capital inicial ao final do período, o Q-Learning mostrou um crescimento muito mais expressivo, alcançando aproximadamente 15 vezes o capital inicial. Isso evidencia que o algoritmo conseguiu identificar e explorar padrões no comportamento da ação, ajustando sua política de negociação para aproveitar melhor as oscilações de preço.

Além disso, nota-se que a curva azul apresenta flutuações mais acentuadas do que a curva laranja, refletindo o risco maior associado a uma estratégia ativa. Ainda assim, o resultado líquido foi superior em termos de rentabilidade acumulada.

O gráfico demonstra que o Q-Learning pode superar significativamente a estratégia de Buy & Hold em termos de retorno sobre o capital investido, sobretudo em mercados voláteis. No entanto, esse desempenho maior vem acompanhado de maior variabilidade nos resultados parciais, o que exige atenção ao risco e à robustez do modelo treinado.

15.5. Evolução do Drawdown ao Longo do Tempo

A Figura-15.4 mostra a evolução do drawdown ao longo do tempo durante o período de teste, comparando as estratégias Q-Learning e Buy & Hold.

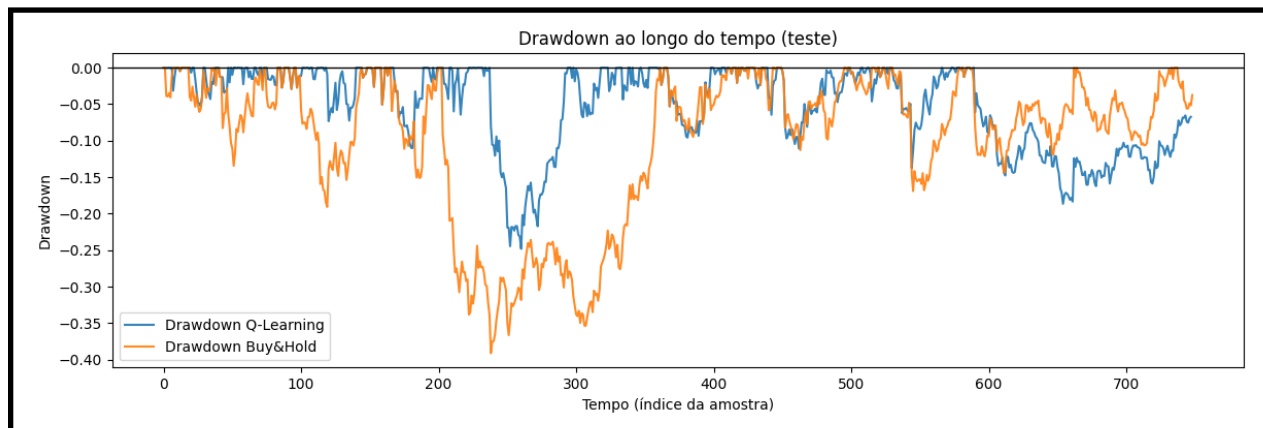


Figura 15.4: Evolução do Drawdown

O drawdown mede a queda percentual do capital em relação ao seu valor máximo histórico, sendo um indicador essencial de risco. Valores próximos de zero significam que o capital está em seu topo histórico, enquanto valores negativos indicam perdas em relação ao máximo já alcançado.

No gráfico, a linha azul representa o drawdown da estratégia Q-Learning, e a linha laranja corresponde ao Buy & Hold. Observa-se que a estratégia Buy & Hold apresenta quedas mais profundas e prolongadas, chegando a cerca de -40% em determinados momentos. Já o Q-Learning apresenta quedas menores, geralmente limitadas a algo em torno de -20%, mostrando maior controle do risco.

Isso indica que o Q-Learning, além de ter potencializado os ganhos (como visto no gráfico de capital acumulado), também conseguiu reduzir as perdas máximas durante os períodos adversos, evidenciando maior robustez e resiliência. Em contrapartida, o Buy & Hold, por manter a posição sem ajustes, sofre diretamente os impactos das oscilações do mercado, refletindo em quedas mais acentuadas do capital.

Este gráfico mostra que o Q-Learning não apenas maximiza os retornos, mas também mitiga riscos, mantendo drawdowns mais baixos em comparação ao Buy & Hold, o que o torna uma estratégia mais eficiente do ponto de vista de risco-retorno.

15.6. Desempenho Q-Learning vs Buy & Hold

Os resultados obtidos mostram uma comparação clara entre a estratégia Q-Learning e a estratégia tradicional de Buy & Hold, usando como base os dados históricos da PETR4.SA. No gráfico de capital acumulado, observa-se que o Q-Learning apresentou um crescimento significativamente maior ao longo do tempo, alcançando valores muito acima da linha de Buy & Hold. Isso demonstra a capacidade do algoritmo em identificar oportunidades de compra e venda mais favoráveis, ajustando-se dinamicamente ao comportamento do mercado. Enquanto o Buy & Hold teve uma evolução estável, porém modesta, o Q-Learning multiplicou de forma expressiva o capital inicial, evidenciando sua superioridade em termos de retorno.

No entanto, retorno não é o único fator a ser considerado em uma estratégia de investimento — o risco também é determinante. Ao analisar o gráfico de drawdown, percebe-se que o Q-Learning apresentou quedas menores e mais controladas, geralmente limitadas a aproximadamente -20%, enquanto o Buy & Hold sofreu quedas mais profundas, chegando a quase -40% em alguns períodos. Isso evidencia que o Q-Learning, além de oferecer retornos mais altos, conseguiu reduzir o impacto das perdas em momentos de mercado adverso, o que se traduz em maior robustez e resiliência.

15.7. Conclusão Integrada

A combinação dos dois resultados mostra que o Q-Learning supera o Buy & Hold tanto em retorno quanto em controle de risco. Ele não apenas maximiza o crescimento do capital, mas também protege melhor contra perdas severas. Isso resulta em uma relação risco-retorno mais vantajosa, tornando-o uma estratégia mais eficiente para cenários de negociação adaptativa.

Assim, enquanto o Buy & Hold permanece exposto às oscilações completas do mercado, o Q-Learning aprende políticas que equilibram ganhos e reduzem perdas, consolidando-se como uma abordagem promissora em aplicações de finanças com aprendizado por reforço.

16

Conclusão do Estudo

16.1. Principais Aprendizados

O Q-learning revela-se eficaz ao obter a política ideal em ambientes markovianos, nos quais o estado subsequente é definido unicamente pelo estado atual e pela ação realizada, mesmo sem um modelo do ambiente.

Contudo, existem limitações: em cenários que englobam vastos espaços de estados ou ações, a tabela Q pode se tornar impraticável, demandando métodos como o Deep Q-Learning, que utiliza redes neurais para calcular a função Q. Além disso, o algoritmo pode revelar sensibilidade quanto à escolha de hiperparâmetros (α , γ , ϵ) e à eficiência da exploração.

Em resumo, o Q-learning se apresenta como um algoritmo eficiente e de fácil entendimento no âmbito do aprendizado por reforço, permitindo que agentes elaborem políticas ótimas por meio da interação com o ambiente, ao proceder com a atualização de uma tabela de valores Q baseada nas recompensas recebidas e nas previsões de recompensas futuras.

A sua simplicidade de utilização torna-o adequado para problemas discretos, enquanto suas extensões, como o Deep Q-Learning, ampliam sua aplicabilidade em contextos mais complexos.

Referências

- [1] Andrew G. Barto. «Intrinsic motivation and reinforcement learning». Em: *Intrinsic Motivation and Reinforcement Learning* 10.11 (2013), pp. 17–47.
- [2] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [3] Richard J. Boucherie e Nico M. van Dijk, eds. *Markov Decision Processes in Practice*. Vol. 248. International Series in Operations Research & Management Science. Cham: Springer, 2017. ISBN: 978-3-319-47764-0. DOI: 10.1007/978-3-319-47766-4.
- [4] Yuxi Li. *Deep Reinforcement Learning*. 2018. arXiv: 1810.06339 [cs.LG]. URL: <https://arxiv.org/abs/1810.06339>.
- [5] Yuxi Li. *Reinforcement Learning Applications*. arXiv preprint arXiv:1908.06973. 2019.
- [6] Volodymyr Mnih et al. «Human-level control through deep reinforcement learning». Em: *Nature* 518.7540 (2015), pp. 529–533.
- [7] Richard S. Sutton e Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT Press, 2018. ISBN: 9780262039246. URL: <http://incompleteideas.net/book/the-book-2nd.html>.